

Learned closures and nonlinear manifolds for parametric projection-based reduced models

S. Ares de Parga, H. Ballout, Y. Maday, H. Hassan

September 11, 2026

Abstract

We compare learned closures and nonlinear manifolds for parametric projection-based reduced-order models of a nonlinear time-dependent system. All models use a common Euclidean proper-orthogonal-decomposition (POD) coordinate system and are trained from full-residual linear projection-based reduced-order model (PROM) trajectories. The comparison includes state-conditioned, parameter-time-conditioned, and hybrid PROM-ANN closures; an intrusive PROM-POD-AE decoder manifold; a non-intrusive POD-NN-ROM direct coefficient map; and a non-intrusive latent POD-DL-ROM.

The numerical study uses a parametric two-dimensional inviscid Burgers benchmark. The intrusive models all minimize the same time-discrete residual, but over different learned trial maps. A shared parameter-time map gives a controlled sweep from a non-intrusive POD-NN-ROM endpoint to the full-coordinate linear PROM. A complementary tail-perturbation diagnostic quantifies how a prescribed Case-2 secondary-coordinate error transfers to the residual-solved primary coordinates. We then augment the prescribed tail with a small residual-adaptive correction space. Its directions follow from eliminating the primary increment in the linearized LSPG problem and are held fixed during each nonlinear time-step solve. With three correction directions and the unchanged baseline network, the mean in-domain state error decreases from 1.617% to 0.447% in a common-initialization comparison. A nested coefficient-data study with 9, 9 + 8, 9 + 12, and 9 + 18 PROM trajectories separates in-domain improvement from extrapolation behaviour. We also state the empirical cubature method (ECM) formulation required for a future HPROM extension; all numerical results reported here are full-residual PROM results.

1 Introduction

Projection-based model order reduction (PMOR) reduces a high-dimensional model while retaining its governing residual and its parameter dependence [1, 2]. For nonlinear, transport-dominated systems, an accurate linear subspace may nevertheless require many coordinates. Nonlinear trial manifolds, learned closures, and latent representations seek to reduce the number of online unknowns without discarding the projection principle [3, 4, 5].

This work compares intrusive closure-enhanced PROMs (PROM-ANN), an intrusive decoder manifold (PROM-POD-AE), a non-intrusive direct coefficient map (POD-NN-ROM), and a non-intrusive latent map (POD-DL-ROM). PROM means that the full time-discrete residual is minimized online. It is not synonymous with a generic ROM: the non-intrusive baselines do not evaluate a residual online. Likewise, the HPROM formulation below is a sampled-residual extension of PROM, not a second name for the same computation.

The reported campaign uses a standard Euclidean POD basis and a common full-residual linear PROM teacher. This permits a direct comparison between a parameter-time-conditioned closure, introduced below as Case 2, which prescribes a learned secondary-coordinate tail but solves a primary coordinate block, and POD-NN-ROM, which predicts the full coordinate vector from the same parameter-time map. Hyperreduction is developed explicitly because an ECM rule must be tied to the trial manifold and its LSPG tangent; HPROM numerical results are deliberately deferred until the corresponding campaign is available.

Two responses to limited regression accuracy are examined separately. The first adds a small number of residual-solved secondary correction amplitudes to Case 2, using the existing trained map. The second enriches the linear-PROM coefficient data and retrain all learned families. The former spends additional work online; the latter spends additional work offline. Their separate assessment identifies what can be recovered from the governing equations before generating further training trajectories.

2 Projection Framework

2.1 Time-discrete residual and Euclidean POD basis

Consider a parametric nonlinear semi-discrete initial-value problem

$$\mathbf{M}(\boldsymbol{\mu})\dot{\mathbf{u}}(t; \boldsymbol{\mu}) + \mathbf{f}(\mathbf{u}(t; \boldsymbol{\mu}); \boldsymbol{\mu}) - \mathbf{g}(t; \boldsymbol{\mu}) = \mathbf{0}, \quad \mathbf{u}(0; \boldsymbol{\mu}) = \mathbf{u}_0(\boldsymbol{\mu}), \quad (1)$$

with $\boldsymbol{\mu} \in \mathcal{D}$. After a chosen implicit time discretization, let $\mathbf{r}^{m+1}(\mathbf{u}; \boldsymbol{\mu}) \in \mathbb{R}^N$ denote the residual at step $m+1$, with dependence on the previous state understood. The HDM solves

$$\mathbf{r}^{m+1}(\mathbf{u}^{m+1}; \boldsymbol{\mu}) = \mathbf{0}. \quad (2)$$

Let $\mathbf{u}_{\text{ref}}$ be the snapshot mean and let $\mathbf{S}_c$ collect the centered HDM snapshots. The thin singular-value decomposition

$$\mathbf{S}_c = \mathbf{U}\boldsymbol{\Sigma}\mathbf{Y}^T \quad (3)$$

defines the Euclidean POD basis retained in this work,

$$\mathbf{V}_{\text{tot}} = \mathbf{U}(:, 1 : n_{\text{tot}}), \quad \mathbf{V}_{\text{tot}}^T \mathbf{V}_{\text{tot}} = \mathbf{I}. \quad (4)$$

The common affine linear trial manifold is

$$\phi_N(\mathbf{q}_N) = \mathbf{u}_{\text{ref}} + \mathbf{V}_{\text{tot}} \mathbf{q}_N, \quad \mathbf{q}_N \in \mathbb{R}^{n_{\text{tot}}}. \quad (5)$$

All learned trial maps below are expressed in these same coordinates.

2.2 Linear PROM and LSPG reference

The full-residual linear PROM is the time-discrete least-squares Petrov–Galerkin (LSPG) problem

$$\mathbf{q}_N^{m+1}(\boldsymbol{\mu}) = \arg \min_{\mathbf{x} \in \mathbb{R}^{n_{\text{tot}}}} \|\mathbf{r}^{m+1}(\phi_N(\mathbf{x}); \boldsymbol{\mu})\|_2^2. \quad (6)$$

At Gauss–Newton iterate k , define

$$\mathbf{T}_N = \mathbf{V}_{\text{tot}}, \quad \mathbf{J}^{m+1,k} = \left. \frac{\partial \mathbf{r}^{m+1}}{\partial \mathbf{u}} \right|_{\phi_N(\mathbf{q}_N^{m+1,k})}, \quad \mathbf{W}_N^{m+1,k} = \mathbf{J}^{m+1,k} \mathbf{T}_N. \quad (7)$$

The normal equations for the increment are

$$\left(\mathbf{W}_N^{m+1,k}\right)^T \mathbf{W}_N^{m+1,k} \Delta \mathbf{q}_N^{m+1,k} = -\left(\mathbf{W}_N^{m+1,k}\right)^T \mathbf{r}^{m+1}\left(\phi_N(\mathbf{q}_N^{m+1,k}); \boldsymbol{\mu}\right). \quad (8)$$

Equation (6) generates all training targets in the PROM campaign.

2.3 Linear HPROM and ECM formulation

The linear HPROM replaces full residual evaluations in the LSPG update by a sparse, positive empirical-cubature approximation. This formulation is given in detail because the same projected-residual algebra is used when the affine trial manifold is replaced by a learned one. Let $\mathcal{E} = \{e_i\}_{i=1}^{N_e}$ be the mesh entities. The time-discrete residual is assembled from local contributions as

$$\mathbf{r}^{m+1}(\mathbf{u}; \boldsymbol{\mu}) = \sum_{e_i \in \mathcal{E}} \mathbf{L}_{e_i}^T \mathbf{r}_{e_i}^{m+1}(\mathbf{L}_{e_i^+} \mathbf{u}; \boldsymbol{\mu}). \quad (9)$$

Here $\mathbf{r}_{e_i}^{m+1} \in \mathbb{R}^{d_{e_i}}$ is the contribution of entity e_i , $\mathbf{L}_{e_i} \in \{0, 1\}^{d_{e_i} \times N}$ extracts its residual degrees of freedom, and $\mathbf{L}_{e_i^+}$ extracts the enlarged state stencil needed to evaluate it. The stencil contains neighbouring entities whenever the discretization requires them.

For the affine trial map (5), the projected LSPG residual at a Gauss–Newton iterate has the entity-wise form

$$\begin{aligned} \mathbf{r}_N^{m+1,k}(\boldsymbol{\mu}) &:= \left(\mathbf{W}_N^{m+1,k} \right)^T \mathbf{r}^{m+1} \left(\phi_N(\mathbf{q}_N^{m+1,k}); \boldsymbol{\mu} \right) \\ &= \sum_{e_i \in \mathcal{E}} \left(\mathbf{L}_{e_i} \mathbf{W}_N^{m+1,k} \right)^T \mathbf{r}_{e_i}^{m+1} \left(\mathbf{L}_{e_i^+} \phi_N(\mathbf{q}_N^{m+1,k}); \boldsymbol{\mu} \right). \end{aligned} \quad (10)$$

Let $\mathbf{J}_{e_i}^{m+1,k}$ be the derivative of the local residual with respect to the stencil state at the same iterate. The Gauss–Newton matrix is assembled analogously,

$$\mathbf{J}_N^{m+1,k} := \left(\mathbf{W}_N^{m+1,k} \right)^T \mathbf{J}^{m+1,k} \mathbf{T}_N = \sum_{e_i \in \mathcal{E}} \left(\mathbf{L}_{e_i} \mathbf{W}_N^{m+1,k} \right)^T \mathbf{J}_{e_i}^{m+1,k} \left(\mathbf{L}_{e_i^+} \mathbf{T}_N \right). \quad (11)$$

ECM selects a reduced entity set $\tilde{\mathcal{E}}_N \subset \mathcal{E}$ and positive weights ξ_{N,e_i} . The same rule approximates both projected quantities,

$$\begin{aligned} \hat{\mathbf{r}}_N^{m+1,k} &= \sum_{e_i \in \tilde{\mathcal{E}}_N} \xi_{N,e_i} \left(\mathbf{L}_{e_i} \mathbf{W}_N^{m+1,k} \right)^T \mathbf{r}_{e_i}^{m+1} \left(\mathbf{L}_{e_i^+} \phi_N(\mathbf{q}_N^{m+1,k}); \boldsymbol{\mu} \right), \\ \hat{\mathbf{J}}_N^{m+1,k} &= \sum_{e_i \in \tilde{\mathcal{E}}_N} \xi_{N,e_i} \left(\mathbf{L}_{e_i} \mathbf{W}_N^{m+1,k} \right)^T \mathbf{J}_{e_i}^{m+1,k} \left(\mathbf{L}_{e_i^+} \mathbf{T}_N \right). \end{aligned} \quad (12)$$

The online sampled update is consequently

$$\hat{\mathbf{J}}_N^{m+1,k} \Delta \mathbf{q}_N^{m+1,k} = -\hat{\mathbf{r}}_N^{m+1,k}, \quad \mathbf{q}_N^{m+1,k+1} = \mathbf{q}_N^{m+1,k} + \Delta \mathbf{q}_N^{m+1,k}. \quad (13)$$

Offline cubature construction. For every selected parameter–time training state s , the linear coordinate $\mathbf{q}_{N,s}$ and its LSPG test basis $\mathbf{W}_{N,s}$ define the contribution of entity e_i to the projected residual,

$$\mathbf{c}_{N,e_i}^{(s)} = (\mathbf{L}_{e_i} \mathbf{W}_{N,s})^T \mathbf{r}_{e_i}(\mathbf{L}_{e_i^+} \phi_N(\mathbf{q}_{N,s}); \boldsymbol{\mu}_s). \quad (14)$$

Stacking all n_{tot} -component contributions over N_{ECM} selected states gives

$$\mathbf{C}_N = \begin{bmatrix} \mathbf{c}_{N,e_1}^{(1)} & \cdots & \mathbf{c}_{N,e_{N_e}}^{(1)} \\ \vdots & & \vdots \\ \mathbf{c}_{N,e_1}^{(N_{\text{ECM}})} & \cdots & \mathbf{c}_{N,e_{N_e}}^{(N_{\text{ECM}})} \end{bmatrix} \in \mathbb{R}^{n_{\text{tot}} N_{\text{ECM}} \times N_e}, \quad \mathbf{b}_N = \mathbf{C}_N \mathbf{1}. \quad (15)$$

ECM seeks a sparse nonnegative approximation of the full projected-residual data,

$$\boldsymbol{\xi}_N = \arg \min_{\boldsymbol{\zeta} \geq \mathbf{0}} \|\mathbf{C}_N \boldsymbol{\zeta} - \mathbf{b}_N\|_2^2, \quad \tilde{\mathcal{E}}_N = \{e_i \in \mathcal{E} : \xi_{N,e_i} > 0\}. \quad (16)$$

In the present implementation, a direct SVD removes numerically null directions of $\mathbf{C}_N$ before the nonnegative cubature solve. The rule is therefore tied to the affine trial manifold and its LSPG test basis. In particular, replacing ϕ_N by a learned closure or decoder also changes the tangent, the test basis, and the projected contributions; the corresponding ECM rule must be constructed for that map.

Relation to other hyperreduction methods. The common step is to approximate an entity-wise sum of *projected* residual contributions, rather than the state reconstruction or a learned map. We use ECM in the sense of empirical cubature [6, 7]. ECSW is another project–then–approximate hyperreduction strategy [8]. It could be used with the same LSPG and trial manifold formulations, but ECM is the rule used in this work.

3 Learned Reduced Representations

3.1 PROM–ANN closures

Partition the common POD basis and coordinate vector as

$$\mathbf{V}_{\text{tot}} = [\mathbf{V} \ \overline{\mathbf{V}}], \quad \mathbf{q}_N = \begin{bmatrix} \mathbf{q} \\ \overline{\mathbf{q}} \end{bmatrix}, \quad n + \bar{n} = n_{\text{tot}}. \quad (17)$$

Here $\mathbf{q} \in \mathbb{R}^n$ and $\overline{\mathbf{q}} \in \mathbb{R}^{\bar{n}}$ denote the primary and secondary coordinates, respectively. The Euclidean POD construction gives

$$\mathbf{V}^T \mathbf{V} = \mathbf{I}_n, \quad \overline{\mathbf{V}}^T \overline{\mathbf{V}} = \mathbf{I}_{\bar{n}}, \quad \mathbf{V}^T \overline{\mathbf{V}} = \mathbf{0}, \quad (18)$$

and the associated linear state is

$$\mathbf{u} = \mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q} + \overline{\mathbf{V}}\overline{\mathbf{q}}. \quad (19)$$

The full-coordinate linear PROM solves both blocks. A learned closure instead supplies the secondary block and leaves only the primary block as an online unknown. PROM–ANN supplies the secondary block through a learned relation

$$\overline{\mathbf{q}} = \mathcal{F}(\mathbf{q}, \boldsymbol{\mu}, t), \quad \phi_{\mathcal{F}}(\mathbf{q}; \boldsymbol{\mu}, t) = \mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q} + \overline{\mathbf{V}}\mathcal{F}(\mathbf{q}, \boldsymbol{\mu}, t). \quad (20)$$

The network provides reduced secondary coordinates, not a direct HDM state approximation. The primary coordinates remain online unknowns of a residual minimization. This is the neural-network-augmented projection construction of [3]; analogous closure roles have also been represented by non-neural regressors [9].

The three cases are information patterns in this common construction:

$$\text{Case 1: } \overline{\mathbf{q}} = \mathcal{N}(\mathbf{q}), \quad \text{Case 2: } \overline{\mathbf{q}} = \mathcal{M}(\boldsymbol{\mu}, t), \quad \text{Case 3: } \overline{\mathbf{q}} = \mathcal{H}(\mathbf{q}, \boldsymbol{\mu}, t). \quad (21)$$

Case 1 is state-conditioned, Case 2 parameter–time-conditioned, and Case 3 combines both inputs. The LSPG problem is

$$\mathbf{q}^{m+1}(\boldsymbol{\mu}) = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{r}^{m+1}(\phi_{\mathcal{F}}(\mathbf{x}; \boldsymbol{\mu}, t^{m+1}); \boldsymbol{\mu})\|_2^2, \quad (22)$$

with tangent

$$\mathbf{T}_{\mathcal{F}} = \mathbf{V} + \overline{\mathbf{V}} \frac{\partial \mathcal{F}}{\partial \mathbf{q}}, \quad \mathbf{W}_{\mathcal{F}} = \mathbf{J} \mathbf{T}_{\mathcal{F}}. \quad (23)$$

In particular, at a Gauss–Newton iterate the coordinate increment solves

$$\Delta \mathbf{q} = \arg \min_{\mathbf{d} \in \mathbb{R}^n} \|\mathbf{r} + \mathbf{J} \mathbf{T}_{\mathcal{F}} \mathbf{d}\|_2^2, \quad \mathbf{W}_{\mathcal{F}}^T \mathbf{W}_{\mathcal{F}} \Delta \mathbf{q} = -\mathbf{W}_{\mathcal{F}}^T \mathbf{r}. \quad (24)$$

The residual, Jacobian, and tangent are all evaluated at that iterate; the previous time-discrete state remains fixed during the nonlinear solve. The normal equations express the optimality condition, while the numerical implementation solves the least-squares system directly. For Case 2, $\partial\mathcal{M}/\partial\mathbf{q} = \mathbf{0}$ and $\mathbf{T}_{\mathcal{M}} = \mathbf{V}$. Its tangent is therefore the cheapest, but its prescribed secondary block is not corrected through a state-dependent tangent.

3.2 PROM–POD–AE and intrusive trial maps

An autoencoder maps the complete linear coordinate vector through a latent variable,

$$\mathbf{z} = \mathcal{E}(\mathbf{q}_N), \quad \hat{\mathbf{q}}_N = \mathcal{D}(\mathbf{z}), \quad \mathbf{z} \in \mathbb{R}^{n_z}. \quad (25)$$

PROM–POD–AE uses the decoder as an intrusive nonlinear trial manifold,

$$\phi_{\text{AE}}(\mathbf{z}) = \mathbf{u}_{\text{ref}} + \mathbf{V}_{\text{tot}}\mathcal{D}(\mathbf{z}), \quad \mathbf{T}_{\text{AE}} = \mathbf{V}_{\text{tot}} \frac{\partial\mathcal{D}}{\partial\mathbf{z}}. \quad (26)$$

Its latent coordinate minimizes the same residual as (22), but over $\mathbf{z}$ rather than $\mathbf{q}$. This is distinct from POD–DL–ROM, which is non-intrusive.

Table 1: Trial decoders and tangent bases of the intrusive models. Every row minimizes the same time-discrete residual over its displayed online variable.

Model	Online variable	Trial decoder	Tangent
Linear PROM/HPROM	$\mathbf{q}_N$	$\mathbf{u}_{\text{ref}} + \mathbf{V}_{\text{tot}}\mathbf{q}_N$	$\mathbf{V}_{\text{tot}}$
PROM–ANN Case 1	$\mathbf{q}$	$\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q} + \bar{\mathbf{V}}\mathcal{N}(\mathbf{q})$	$\mathbf{V} + \bar{\mathbf{V}} \partial\mathcal{N}/\partial\mathbf{q}$
PROM–ANN Case 2	$\mathbf{q}$	$\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q} + \bar{\mathbf{V}}\mathcal{M}(\boldsymbol{\mu}, t)$	$\mathbf{V}$
PROM–ANN Case 3	$\mathbf{q}$	$\mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q} + \bar{\mathbf{V}}\mathcal{H}(\mathbf{q}, \boldsymbol{\mu}, t)$	$\mathbf{V} + \bar{\mathbf{V}} \partial\mathcal{H}/\partial\mathbf{q}$
PROM–POD–AE	$\mathbf{z}$	$\mathbf{u}_{\text{ref}} + \mathbf{V}_{\text{tot}}\mathcal{D}(\mathbf{z})$	$\mathbf{V}_{\text{tot}} \partial\mathcal{D}/\partial\mathbf{z}$

3.3 Model-specific ECM construction

The HPROM extension does not reuse the linear rule in (16). For any intrusive map ϕ_c and tangent $\mathbf{T}_c$, the entity contributions use $\mathbf{W}_c = \mathbf{J}\mathbf{T}_c$. A manifold coordinate is first fitted at every selected HDM state,

$$\mathbf{y}_{c,s} = \arg \min_{\mathbf{y}} \|\mathbf{u}_s^{m_s+1} - \phi_c(\mathbf{y}; \boldsymbol{\mu}_s, t^{m_s+1})\|_2^2. \quad (27)$$

For Case 2 this is linear least squares after subtracting the prescribed tail,

$$\mathbf{q}_{2,s} = \mathbf{V}^\dagger [\mathbf{u}_s^{m_s+1} - \mathbf{u}_{\text{ref}} - \bar{\mathbf{V}}\mathcal{M}(\boldsymbol{\mu}_s, t^{m_s+1})]. \quad (28)$$

For Cases 1 and 3 and for the decoder manifold, it is nonlinear least squares, for example with the Gauss–Newton update

$$\mathbf{y}_{c,s}^{(k+1)} = \mathbf{y}_{c,s}^{(k)} - \left[\frac{\partial\phi_c}{\partial\mathbf{y}} \right]^\dagger \left[\phi_c(\mathbf{y}_{c,s}^{(k)}; \boldsymbol{\mu}_s, t^{m_s+1}) - \mathbf{u}_s^{m_s+1} \right]. \quad (29)$$

At the fitted state,

$$\mathbf{c}_{c,e_i}^{(s)} = (\mathbf{L}_{e_i} \mathbf{W}_{c,s})^T \mathbf{r}_{e_i}(\mathbf{L}_{e_i}^\dagger \phi_c(\mathbf{y}_{c,s}; \boldsymbol{\mu}_s, t^{m_s+1}); \boldsymbol{\mu}_s). \quad (30)$$

Stacking these vectors produces $\mathbf{C}_c$ and $\mathbf{b}_c = \mathbf{C}_c \mathbf{1}$ as in (16); a positive ECM rule is then computed for each map. The trained closure or decoder, its tangent, and the parameter–time sample set are therefore part of a rule. A linear-HPROM rule, or one built for a different nonlinear map, cannot be reused [6, 7, 10].

3.4 Non-intrusive reference models

POD–NN–ROM is a direct map

$$\mathbf{q}_N(t; \boldsymbol{\mu}) \approx \mathcal{G}_q(\boldsymbol{\mu}, t), \quad (31)$$

and does not solve a residual online. POD–DL–ROM uses a learned latent map and decoder,

$$\mathbf{z}(t; \boldsymbol{\mu}) \approx \mathcal{G}_z(\boldsymbol{\mu}, t), \quad \mathbf{q}_N(t; \boldsymbol{\mu}) \approx \mathcal{D}(\mathcal{G}_z(\boldsymbol{\mu}, t)), \quad (32)$$

following the POD–DL–ROM principle [11]. These are direct reduced-coordinate regressors, not PROM residual solves.

4 Residual-Adaptive Secondary Correction for Case 2

4.1 From a prescribed tail to a corrected trial space

In Case 2, the network fixes $\bar{\mathbf{q}} = \mathcal{M}(\boldsymbol{\mu}, t)$ and LSPG adjusts only $\mathbf{q}$. At a fixed linearization, primary increments change the residual only within $\text{range}(\mathbf{J}\mathbf{V})$. The component of a secondary prediction error’s linearized residual contribution orthogonal to that space cannot be removed by primary increments alone. The proposed extension retains the trained map and permits selected secondary corrections online.

At step $m + 1$, let $\mathbf{B}_r^{m+1} \in \mathbb{R}^{\bar{n} \times r}$ contain r orthonormal directions in secondary-coordinate space, where $0 \leq r \leq \bar{n}$. Their construction is given below. Introduce r additional amplitudes $\mathbf{a} \in \mathbb{R}^r$ and define

$$\begin{aligned} \bar{\mathbf{q}} &= \mathcal{M}(\boldsymbol{\mu}, t^{m+1}) + \mathbf{B}_r^{m+1} \mathbf{a}, \\ \phi_r^{m+1}(\mathbf{q}, \mathbf{a}; \boldsymbol{\mu}) &= \mathbf{u}_{\text{ref}} + \mathbf{V}\mathbf{q} + \bar{\mathbf{V}} [\mathcal{M}(\boldsymbol{\mu}, t^{m+1}) + \mathbf{B}_r^{m+1} \mathbf{a}]. \end{aligned} \quad (33)$$

Each column of $\mathbf{B}_r^{m+1}$ is a pattern over all $\bar{n}$ secondary coordinates. Thus r counts independent correction patterns, not selected POD coefficients. A single amplitude can modify every secondary coordinate through its corresponding column.

The complete coordinate vector retains the notation of the linear reference:

$$\mathbf{q}_N^{m+1} = \left[\mathcal{M}(\boldsymbol{\mu}, t^{m+1}) + \mathbf{B}_r^{m+1} \mathbf{a}^{m+1} \right] \in \mathbb{R}^{n_{\text{tot}}}, \quad \mathbf{u}_r^{m+1} = \mathbf{u}_{\text{ref}} + \mathbf{V}_{\text{tot}} \mathbf{q}_N^{m+1}. \quad (34)$$

Thus $\mathbf{q}_N$ always denotes the complete reconstructed reduced-coordinate vector; $[\mathbf{q}; \mathbf{a}]$ contains only the independent online unknowns.

Once $\mathbf{B}_r^{m+1}$ has been constructed and held fixed, the map (33) is affine in its $n + r$ unknowns. Its exact tangent is

$$\mathbf{T}_r^{m+1} = [\mathbf{V} \quad \bar{\mathbf{V}} \mathbf{B}_r^{m+1}], \quad (\mathbf{T}_r^{m+1})^T \mathbf{T}_r^{m+1} = \mathbf{I}_{n+r}. \quad (35)$$

The second identity follows from (18) and $(\mathbf{B}_r^{m+1})^T \mathbf{B}_r^{m+1} = \mathbf{I}_r$. It also gives $\|\bar{\mathbf{V}} \mathbf{B}_r^{m+1} \mathbf{a}\|_2 = \|\mathbf{a}\|_2$: the amplitude norm measures the state correction in the Euclidean norm already used by the POD basis.

4.2 Eliminating the primary increment

The correction directions should address residual components that the primary update cannot remove. To identify them, first form the predictor

$$\mathbf{u}_*^{m+1} = \mathbf{u}_{\text{ref}} + \mathbf{V} \mathbf{q}^m + \bar{\mathbf{V}} \mathcal{M}(\boldsymbol{\mu}, t^{m+1}). \quad (36)$$

The residual at this predictor uses the previous *computed* state $\mathbf{u}_r^m$ in its time-discretization terms. No reference trajectory enters that evaluation. In the derivation that follows, suppress $m + 1$ and $\boldsymbol{\mu}$ and write

$$\mathbf{r}_* = \mathbf{r}^{m+1}(\mathbf{u}_*^{m+1}; \boldsymbol{\mu}), \quad \mathbf{J}_* = \left. \frac{\partial \mathbf{r}^{m+1}}{\partial \mathbf{u}} \right|_{\mathbf{u}_*^{m+1}}, \quad \mathbf{A} = \mathbf{J}_* \mathbf{V}. \quad (37)$$

If both coordinate blocks were free, the linearized correction would solve

$$\min_{\Delta \mathbf{q} \in \mathbb{R}^n, \Delta \bar{\mathbf{q}} \in \mathbb{R}^{\bar{n}}} \|\mathbf{r}_* + \mathbf{A} \Delta \mathbf{q} + \mathbf{J}_* \bar{\mathbf{V}} \Delta \bar{\mathbf{q}}\|_2^2. \quad (38)$$

Assume that $\mathbf{A}$ has full column rank and take its thin QR factorization $\mathbf{A} = \mathbf{Q} \mathbf{R}_A$, with $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$. Define the orthogonal projector

$$\mathbf{P} = \mathbf{I}_N - \mathbf{Q} \mathbf{Q}^T. \quad (39)$$

For a fixed $\Delta \bar{\mathbf{q}}$, least squares eliminates the primary increment:

$$\Delta \mathbf{q}(\Delta \bar{\mathbf{q}}) = -\mathbf{A}^\dagger (\mathbf{r}_* + \mathbf{J}_* \bar{\mathbf{V}} \Delta \bar{\mathbf{q}}). \quad (40)$$

Substitution into (38) leaves precisely

$$\min_{\Delta \bar{\mathbf{q}} \in \mathbb{R}^{\bar{n}}} \|\mathbf{P} \mathbf{r}_* + \mathbf{C} \Delta \bar{\mathbf{q}}\|_2^2, \quad \mathbf{C} = \mathbf{P} \mathbf{J}_* \bar{\mathbf{V}}. \quad (41)$$

The projector is essential: a direction is assessed by its effect on the residual *after* the best primary correction, rather than by its effect on a residual component already removable through $\mathbf{V}$.

Define the symmetric positive-semidefinite operator and right-hand side

$$\mathbf{H} = \mathbf{C}^T \mathbf{C}, \quad \mathbf{g} = \mathbf{C}^T \mathbf{P} \mathbf{r}_* = \bar{\mathbf{V}}^T \mathbf{J}_*^T \mathbf{P} \mathbf{r}_*. \quad (42)$$

Then the unconstrained secondary normal equations are $\mathbf{H} \Delta \bar{\mathbf{q}} = -\mathbf{g}$. Equivalently, $\mathbf{H}$ is the Schur complement obtained by eliminating the primary block of the full linearized normal equations. It is positive definite if $\mathbf{J}_* \mathbf{V}_{\text{tot}}$ has full column rank. These operators are derived from the current Euclidean LSPG problem; they do not alter the POD basis or the offline training targets.

4.3 A small Krylov space for the secondary correction

Solving (41) for all $\bar{n}$ coordinates would undo the reduction achieved by the closure. Instead, for $r \geq 1$ and before any Krylov breakdown, choose

$$\text{range}(\mathbf{B}_r) = \mathcal{K}_r(\mathbf{H}, \mathbf{g}) := \text{span}\{\mathbf{g}, \mathbf{H}\mathbf{g}, \dots, \mathbf{H}^{r-1}\mathbf{g}\}, \quad \mathbf{B}_r^T \mathbf{B}_r = \mathbf{I}_r. \quad (43)$$

Here r is a user-selected computational budget. Three directions are one possible choice, not a restriction of the formulation. The construction uses orthogonalized operator applications, so it does not explicitly form powers of $\mathbf{H}$.

For $r = 1$ and $\mathbf{g} \neq \mathbf{0}$, the space is the span of the negative gradient of one half of the objective in (41) at zero secondary increment. The linearized minimizer along this direction is

$$\Delta \bar{\mathbf{q}}_1 = -\frac{\mathbf{g}^T \mathbf{g}}{\mathbf{g}^T \mathbf{H} \mathbf{g}} \mathbf{g}. \quad (44)$$

Indeed, inserting $\Delta \bar{\mathbf{q}} = -\alpha \mathbf{g}$ in (41) and differentiating with respect to α gives (44). Because $\mathbf{g} \in \text{range}(\mathbf{C}^T) = \ker(\mathbf{C})^\perp$, a nonzero $\mathbf{g}$ satisfies $\mathbf{g}^T \mathbf{H} \mathbf{g} = \|\mathbf{C} \mathbf{g}\|_2^2 > 0$; no assumption that

the residual can be reduced to zero is needed. Higher ranks permit an optimal combination of additional directions $\mathbf{H}\mathbf{g}, \mathbf{H}^2\mathbf{g}, \dots$ in the same linearized problem. For a fixed predictor their spaces are nested, hence

$$\min_{\mathbf{a} \in \mathbb{R}^{r+1}} \|\mathbf{P}\mathbf{r}_* + \mathbf{C}\mathbf{B}_{r+1}\mathbf{a}\|_2 \leq \min_{\mathbf{a} \in \mathbb{R}^r} \|\mathbf{P}\mathbf{r}_* + \mathbf{C}\mathbf{B}_r\mathbf{a}\|_2. \quad (45)$$

This is a statement about the linearized residual at the same predictor. It is not an error bound against the HDM and does not imply monotone trajectory accuracy as r increases.

The operator actions can be evaluated without assembling either $\mathbf{C}$ or the $N \times \bar{n}$ product $\mathbf{J}_*\bar{\mathbf{V}}$:

$$\begin{aligned} \mathbf{P}\mathbf{y} &= \mathbf{y} - \mathbf{Q}(\mathbf{Q}^T\mathbf{y}), \\ \mathbf{H}\mathbf{v} &= \bar{\mathbf{V}}^T \mathbf{J}_*^T \mathbf{P} [\mathbf{J}_*(\bar{\mathbf{V}}\mathbf{v})]. \end{aligned} \quad (46)$$

The implementation constructs the orthonormal columns by Arnoldi orthogonalization with two reorthogonalization passes. If $\mathbf{g}$ is numerically zero, no secondary direction is added. If a later vector is numerically dependent, the construction stops and uses the attained rank $r_m \leq r$. In that event all formulas use r_m in place of r ; a request for r directions does not require adding null or dependent ones.

4.4 Nonlinear solve on the frozen affine space

The linearized problem is used to *select directions*. The accepted time-step solution is obtained by minimizing the original nonlinear time-discrete residual on (33):

$$\begin{bmatrix} \mathbf{q}^{m+1} \\ \mathbf{a}^{m+1} \end{bmatrix} = \arg \min_{\mathbf{x} \in \mathbb{R}^n, \mathbf{b} \in \mathbb{R}^{r_m}} \|\mathbf{r}^{m+1}(\phi_r^{m+1}(\mathbf{x}, \mathbf{b}; \boldsymbol{\mu}); \boldsymbol{\mu})\|_2^2. \quad (47)$$

Writing $\mathbf{z} = [\mathbf{q}; \mathbf{a}]$, the Gauss–Newton increment is

$$\Delta \mathbf{z}^{m+1,k} = \arg \min_{\mathbf{d} \in \mathbb{R}^{n+r_m}} \left\| \mathbf{r}^{m+1,k} + \mathbf{J}^{m+1,k} \mathbf{T}_r^{m+1} \mathbf{d} \right\|_2^2. \quad (48)$$

Although $\mathbf{J}^{m+1,k}$ changes with the iterate, $\mathbf{B}_r^{m+1}$ and $\mathbf{T}_r^{m+1}$ remain fixed throughout this solve. Consequently (35) is the exact derivative of the trial map being optimized. The method does not omit a derivative of a state-dependent $\mathbf{B}_r$: constructing the space and solving on that fixed space are two distinct operations.

The complete online step is as follows.

1. Predict $\mathcal{M}(\boldsymbol{\mu}, t^{m+1})$ with the trained network and form the predictor (36) from the preceding primary coordinates $\mathbf{q}^m$.
2. Evaluate $\mathbf{r}_*$ and $\mathbf{J}_*$ using the preceding computed full reduced state $\mathbf{u}_r^m$ in the time-discrete residual.
3. Factor $\mathbf{J}_*\mathbf{V}$, apply (46), and construct up to r columns of $\mathbf{B}_r^{m+1}$ according to (43).
4. Form $\mathbf{T}_r^{m+1}$ and initialize the nonlinear solve with $\mathbf{z}^{m+1,0} = [\mathbf{q}^m; \mathbf{0}]$. Solve (47) by (48).
5. Store $\mathbf{q}^{m+1}$ and the corrected secondary coordinates $\bar{\mathbf{q}}^{m+1} = \mathcal{M}(\boldsymbol{\mu}, t^{m+1}) + \mathbf{B}_r^{m+1}\mathbf{a}^{m+1}$.

Their reconstructed state is the predecessor used at the next time step.

At the next step a new correction space is constructed. Its amplitudes are initialized to zero because the columns generally differ from those at the previous step; the previous correction is nevertheless retained through the time-discrete predecessor state.

4.5 Interpretation, limiting cases, and cost

The method changes the online trial space while retaining the trained parameter–time map. Increasing n would instead free additional individual POD coordinates. Increasing r keeps the same primary/secondary split and frees combinations of secondary coordinates selected by the current residual and Jacobian. The two choices can therefore give different accuracy at the same number of online unknowns.

For $r = 0$, (33) is the original Case–2 map, and its online trajectory is recovered when the initialization and nonlinear solver settings also match. If the constructed columns span the entire secondary space, the corrected affine space equals the full linear-PROM space:

$$\{\phi_n^{m+1}(\mathbf{q}, \mathbf{a}; \boldsymbol{\mu})\} = \mathbf{u}_{\text{ref}} + \text{range}(\mathbf{V}_{\text{tot}}). \quad (49)$$

The network offset then lies inside the freely variable linear space. Equality of converged trajectories additionally requires compatible initial states and selection of the same time-step minimizers. A smaller Krylov space can solve the predictor’s linearized problem exactly upon breakdown, without spanning the full secondary space; that fact does not make the subsequent nonlinear problems identical.

At a fixed predecessor, the uncorrected Case–2 space is contained in the corrected space by taking $\mathbf{a} = \mathbf{0}$. Its global residual minimum therefore cannot increase upon enlargement. In practice, Gauss–Newton is stopped at finite tolerances, and different trajectories produce different predecessor states. Neither space inclusion nor (45) guarantees lower state error over a full trajectory, nonlinear convergence, or superiority to Cases 1 and 3.

Finally, $n + r_m$ unknowns do not imply the cost of a fixed $(n + r_m)$ -mode linear PROM. Construction requires the full residual and Jacobian, a QR factorization of $\mathbf{J}_* \mathbf{V}$, and actions involving all $\bar{n}$ secondary modes. The final nonlinear solves use $\mathbf{J} \mathbf{T}_r$, whose width is only $n + r_m$, and need no network derivative with respect to the primary coordinates. Both the direction-construction cost and the nonlinear solve must be included in online timings. The present construction is a full-residual PROM method; its adaptive spaces have not been equipped with an ECM rule in this study.

5 Benchmark and PROM Protocol

5.1 Burgers problem and discretization

We use the two-dimensional parametric inviscid Burgers benchmark of [3],

$$\frac{\partial \mathbf{U}}{\partial t} + \frac{\partial \mathbf{F}(\mathbf{U})}{\partial x} + \frac{\partial \mathbf{G}(\mathbf{U})}{\partial y} = \mathbf{S}(x; \boldsymbol{\mu}), \quad \mathbf{U} = \begin{bmatrix} u_x \\ u_y \end{bmatrix}, \quad (50)$$

where

$$\mathbf{F}(\mathbf{U}) = \begin{bmatrix} \frac{1}{2} u_x^2 \\ \frac{1}{2} u_x u_y \end{bmatrix}, \quad \mathbf{G}(\mathbf{U}) = \begin{bmatrix} \frac{1}{2} u_x u_y \\ \frac{1}{2} u_y^2 \end{bmatrix}, \quad \mathbf{S}(x; \boldsymbol{\mu}) = \begin{bmatrix} 0.02 \exp(\mu_2 x) \\ 0 \end{bmatrix}. \quad (51)$$

The domain is $[0, 100]^2$, $T_f = 25$, and $\mathcal{D} = [4.25, 5.50] \times [0.015, 0.03]$. The HDM uses a cell-centered finite-volume Godunov discretization on a 250×250 grid ($N = 125,000$) and 500 uniform time steps with $\Delta t = 0.05$.

5.2 Data, validation, and online points

Nine HDM trajectories on a 3×3 tensor grid define the centered snapshot matrix for the Euclidean POD. We retain $n_{\text{tot}} = 151$ modes, which capture 99.9900% of its energy. At these same nine parameters we solve the full-residual linear PROM. The resulting PROM-generated trajectories, rather than HDM snapshots orthogonally projected after the fact, are the learning

targets for every model. This comparison-control choice aligns every learned model with the reduced optimality problem used as its teacher; it is not the conventional purely non-intrusive workflow, which typically uses projected HDM labels.

Two complete 151-coordinate linear-PROM trajectories are reserved before training,

$$\boldsymbol{\mu}^{\text{val},1} = (4.5625, 0.02625), \quad \boldsymbol{\mu}^{\text{val},2} = (5.1875, 0.01875). \quad (52)$$

Because each held-out parameter contains all time samples, this is a parameter-generalization test rather than a random time-snapshot split. The four online evaluations are

$$\begin{aligned} \boldsymbol{\mu}^{(v)} &= (4.875, 0.0225), \quad \boldsymbol{\mu}^{(1)} = (4.560, 0.0190), \\ \boldsymbol{\mu}^{(2)} &= (5.190, 0.0260), \quad \boldsymbol{\mu}^{(3)} = (4.000, 0.0330). \end{aligned} \quad (53)$$

The first three are in-domain and $\boldsymbol{\mu}^{(3)}$ is extrapolatory. The validation locations are neither training nor online evaluation points.

5.3 Nested coefficient-data enrichment

To isolate data coverage from architecture and online formulation, we use a fixed seed-42 Latin-hypercube candidate design over the original parameter box and a 25% coordinate-wise margin. Three deterministic nested subsets are retained. The 9 + 8 level contains the baseline grid, four interior points, and four margin points. The 9 + 12 level adds two interior and two margin points to that set, and the 9 + 18 level adds another three of each. The complete 9 + 36 candidate design defines the nested ordering only; no 9 + 36 PROM coefficient campaign is used in this study. All online evaluation and parameter-validation locations are excluded from the enrichment sets.

The word “margin” denotes the union of the left, right, top, and bottom strips of the expanded rectangle. It does not enforce coverage of their intersections. This distinction matters for the upper-left extrapolation point $\boldsymbol{\mu}^{(3)}$ and will be revisited with the online results.

Table 2: Nested Euclidean-POD PROM coefficient data. The same two complete linear-PROM trajectories are held out for parameter validation at every level.

Quantity	Baseline	9 + 8	9 + 12	9 + 18
Baseline grid trajectories	9	9	9	9
Interior LHS trajectories	0	4	6	9
Margin LHS trajectories	0	4	6	9
Total training trajectories	9	17	21	27
Training samples	4509	8517	10521	13527
Held-out parameter-validation trajectories	2	2	2	2
Held-out parameter-validation samples	1002	1002	1002	1002
LHS seed	–	42	42	42
Margin fraction	–	0.25	0.25	0.25

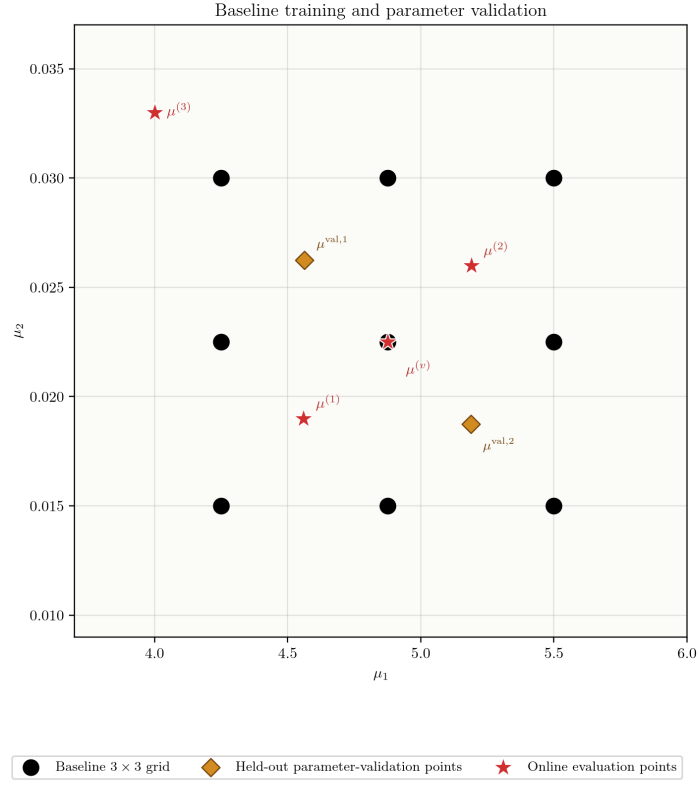


Figure 1: Baseline Euclidean-POD PROM data layout. Black circles are the nine HDM and linear-PROM training parameters; diamonds are the two held-out linear-PROM parameter-validation trajectories; red stars are the four online evaluation points.

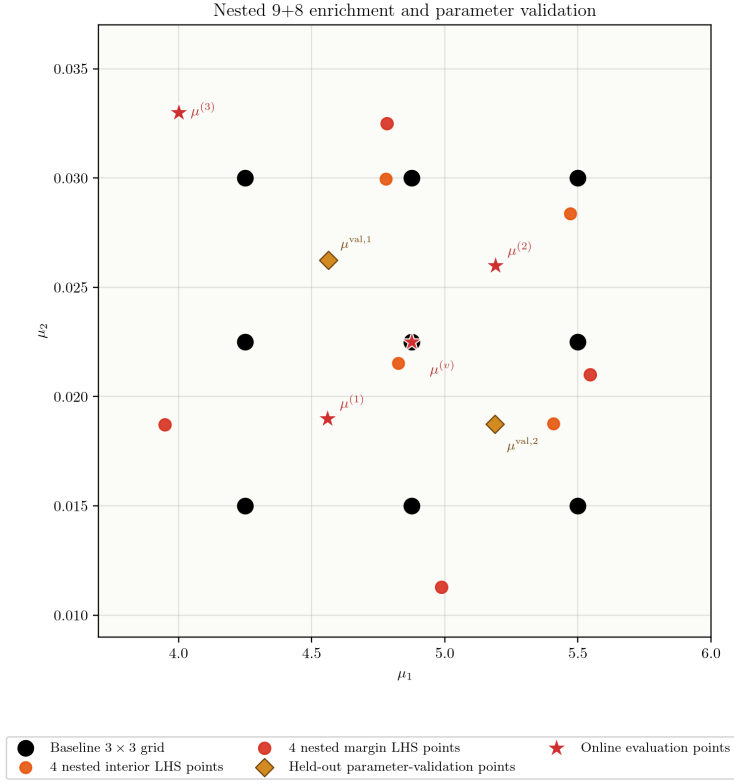


Figure 2: 9 + 8 nested design. Four interior and four margin locations are added to the baseline grid. The extrapolatory point $\boldsymbol{\mu}^{(3)}$ lies in the upper-left corner, whereas the 9 + 8 subset samples its left and top margins separately.

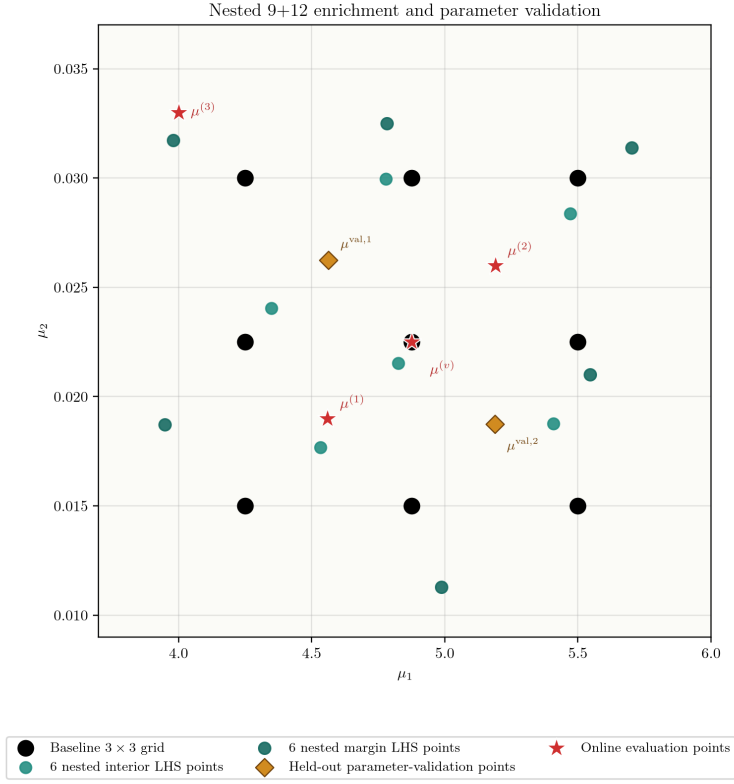


Figure 3: 9 + 12 nested design. Relative to 9 + 8, two interior and two margin locations are added. One of the latter, near $(3.980, 0.03173)$, samples the upper-left parameter-space region close to $\boldsymbol{\mu}^{(3)} = (4.000, 0.0330)$.

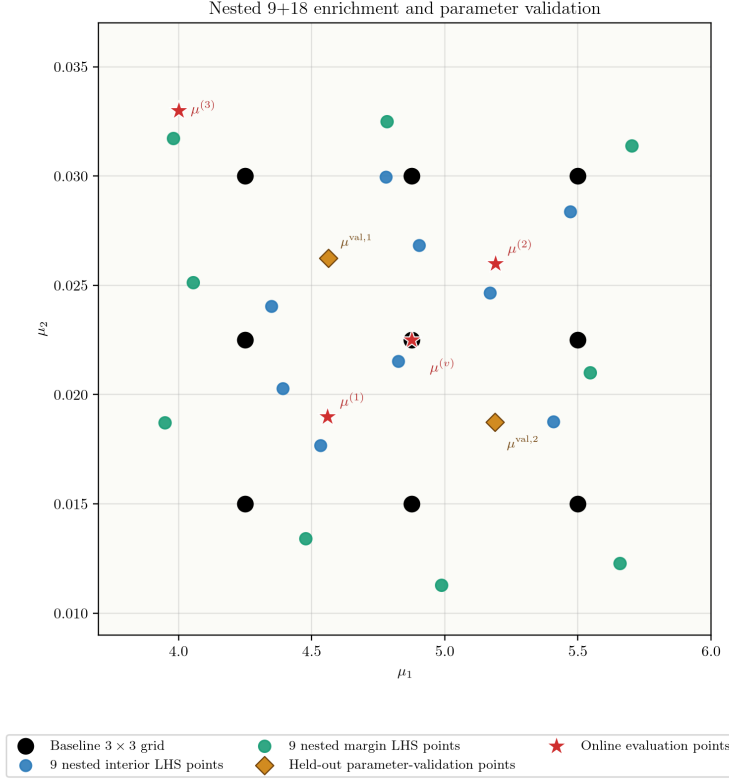


Figure 4: 9 + 18 nested design. It retains all 9 + 12 locations and adds three interior and three margin points, providing the largest parameter-space coverage used in the reported PROM campaign.

5.4 Model training and shared Case-2 map

The baseline networks use 4509 training samples, and the 9 + 8, 9 + 12, and 9 + 18 networks use 8517, 10521, and 13527, respectively. Every network uses the same 1002 external-validation parameter-time samples. Case 1 and Case 3 use $n = 10$ primary coordinates; PROM-POD-AE and POD-DL-ROM use $n_z = 10$. The architectures and reduced coordinate errors are reported in Table 3.

Table 3: Training and held-out parameter-validation errors in reduced coordinates. The master map is shared between the Case-2 tail and the POD-NN-ROM direct endpoint.

Model	Learned map	Train e_q (%)	Validation e_q (%)	Parameters
PROM-ANN Case 1	$\mathbf{q}_p \mapsto \mathbf{q}_s$	1.422	3.590	564,621
Master map (Case 2 / POD-NN-ROM)	$(\boldsymbol{\mu}, t) \mapsto \mathbf{q}_{151}$	2.888	4.261	565,399
PROM-ANN Case 3	$(\mathbf{q}_p, \boldsymbol{\mu}, t) \mapsto \mathbf{q}_s$	0.572	3.482	565,389
PROM-POD-AE	$\mathbf{q}_{151} \mapsto \mathbf{z} \mapsto \hat{\mathbf{q}}_{151}$	0.194	2.673	486,817
POD-DL-ROM	$(\boldsymbol{\mu}, t) \mapsto \mathbf{z} \mapsto \hat{\mathbf{q}}_{151}$	1.843	2.981	952,747

For the controlled Case-2 comparison, one master map is trained as

$$\mathcal{G}_q : (\boldsymbol{\mu}, t) \mapsto \hat{\mathbf{q}}_N \in \mathbb{R}^{n_{\text{tot}}}. \quad (54)$$

For primary dimension n , Case 2 inserts $\bar{\mathbf{q}} = \mathbf{P}_{>n} \mathcal{G}_q(\boldsymbol{\mu}, t)$ into the residual solve, whereas POD-NN-ROM uses the entire vector directly. Thus $n = 0$ is exactly the POD-NN-ROM endpoint

from this same trained map, and $n = 151$ is the linear PROM endpoint. The shared-map construction is a diagnostic control, not a restriction of the general Case-2 formulation.

5.5 Protocol for the residual-adaptive comparison

The correction experiment uses the unchanged baseline master ANN trained from the nine linear-PROM trajectories, with the two validation parameters in (52). It neither generates additional training trajectories nor retraining the network. In particular, the online directions are constructed from the current computed state, residual, and Jacobian, not from HDM or linear-PROM solutions at the evaluation parameters.

A preliminary local screening compared $r = 1, 3, 5$ at the two validation parameters. Their mean full-trajectory coefficient errors against the linear PROM were 1.988%, 1.093%, and 1.123%, respectively. The resulting choice $r = 3$ was frozen before evaluation at the four reporting parameters. These two validation parameters also serve network selection; they are not an independent test set. The rank screening used the original ANN-offset initial representation. The subsequent common-initialization comparison below retained the selected rank rather than selecting it again. Thus $r = 3$ is the tested configuration, not a claim of optimal rank for every initialization or parameter.

The dedicated Sherlock comparison isolates the online correction using a common initial state for all four methods. Specifically,

$$\mathbf{q}_N^0 = \mathbf{V}_{\text{tot}}^T (\mathbf{u}_0 - \mathbf{u}_{\text{ref}}), \quad \mathbf{u}_{\text{ROM}}^0 = \mathbf{u}_{\text{ref}} + \mathbf{V}_{\text{tot}} \mathbf{q}_N^0. \quad (55)$$

This state need not belong to each learned trial manifold at $t = 0$; it is used as the predecessor for the first time step. Subsequent states are computed on the respective trial maps. This uses the prescribed initial condition, not future reference snapshots. The uncorrected Case-2 trajectory is therefore recomputed with this initialization and is not interchanged with the ANN-initialized baseline in Section 6.

Here $n_{\text{tot}} = 151$, $n = 10$, and $\bar{n} = 141$. Consequently, $\mathbf{B}_3^{m+1}$ has size 141×3 and the nonlinear solve has $10 + 3 = 13$ unknowns, while $\mathbf{q}_N$ still has 151 entries. The attained rank was three at every step of all four reported trajectories. All methods use 500 time steps and the same nominal Gauss-Newton settings: at most 20 iterations, a relative residual-norm threshold of 10^{-5} , and a relative residual-change stopping threshold of 10^{-2} . The native implementations are retained: the networks are evaluated in single precision, Cases 1 and 3 use their neural closure/tangent path, and Case 2 and its corrected variant perform the affine reduced solve in double precision. The timings therefore compare these implementations, not operation counts alone.

Four methods at four parameters were run sequentially on the same Sherlock node, equipped with Intel Xeon Gold 5118 processors, with 24 numerical threads. There is one measured full trajectory per method and parameter, and no full-trajectory warmup. The timed interval includes initialization, network prediction, direction and tangent construction when applicable, and time integration; it excludes model/data loading, accuracy evaluation, and result-file writing. The reported mean is a mean over the three in-domain parameters, not over repeated timings. No matched HDM timing is available for this experiment, so no HDM-relative speed-up is inferred.

5.6 Reported errors

For each evaluation parameter, the state error is

$$e_u(\boldsymbol{\mu}) = 100 \frac{\|\mathbf{u}_{\text{HDM}} - \mathbf{u}_{\text{ROM}}\|_F}{\|\mathbf{u}_{\text{HDM}}\|_F}, \quad (56)$$

and the coefficient error against the full linear PROM trajectory is

$$e_q(\boldsymbol{\mu}) = 100 \frac{\|\mathbf{q}_N^{\text{lin}} - \hat{\mathbf{q}}_N\|_F}{\|\mathbf{q}_N^{\text{lin}}\|_F}. \quad (57)$$

The linear PROM is an accuracy reference for the fixed basis and full residual, not a universal lower bound with respect to the HDM.

6 Euclidean-POD PROM Results

6.1 Baseline state accuracy and solution fields

Table 4 reports space–time state errors. The linear PROM remains below 0.5% at the three in-domain points. Among the learned intrusive models, Case 3 and PROM–POD–AE remain closest to that reference in domain. Case 2 improves on the direct POD–NN–ROM endpoint at all four points, but its prescribed tail remains visible, particularly away from the training grid. POD–DL–ROM is the closer of the two non-intrusive baselines in this campaign.

Table 4: Online state errors $100 \|\mathbf{u}_{\text{HDM}} - \mathbf{u}_{\text{ROM}}\|_F / \|\mathbf{u}_{\text{HDM}}\|_F$. The mean averages the three in-domain points. The horizontal rule separates intrusive residual-minimizing models from non-intrusive maps.

Model	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$
Linear PROM	0.410	0.450	0.463	0.441	0.850
PROM–ANN Case 1	0.418	0.543	0.736	0.566	1.673
PROM–ANN Case 2 ($n = 10$)	1.163	1.900	1.758	1.607	2.696
PROM–ANN Case 3	0.408	0.512	0.563	0.494	1.521
PROM–POD–AE ($n_z = 10$)	0.411	0.533	0.711	0.551	1.754
POD–NN–ROM	1.187	1.918	1.989	1.698	4.351
POD–DL–ROM ($n_z = 10$)	0.827	1.178	1.370	1.125	2.267

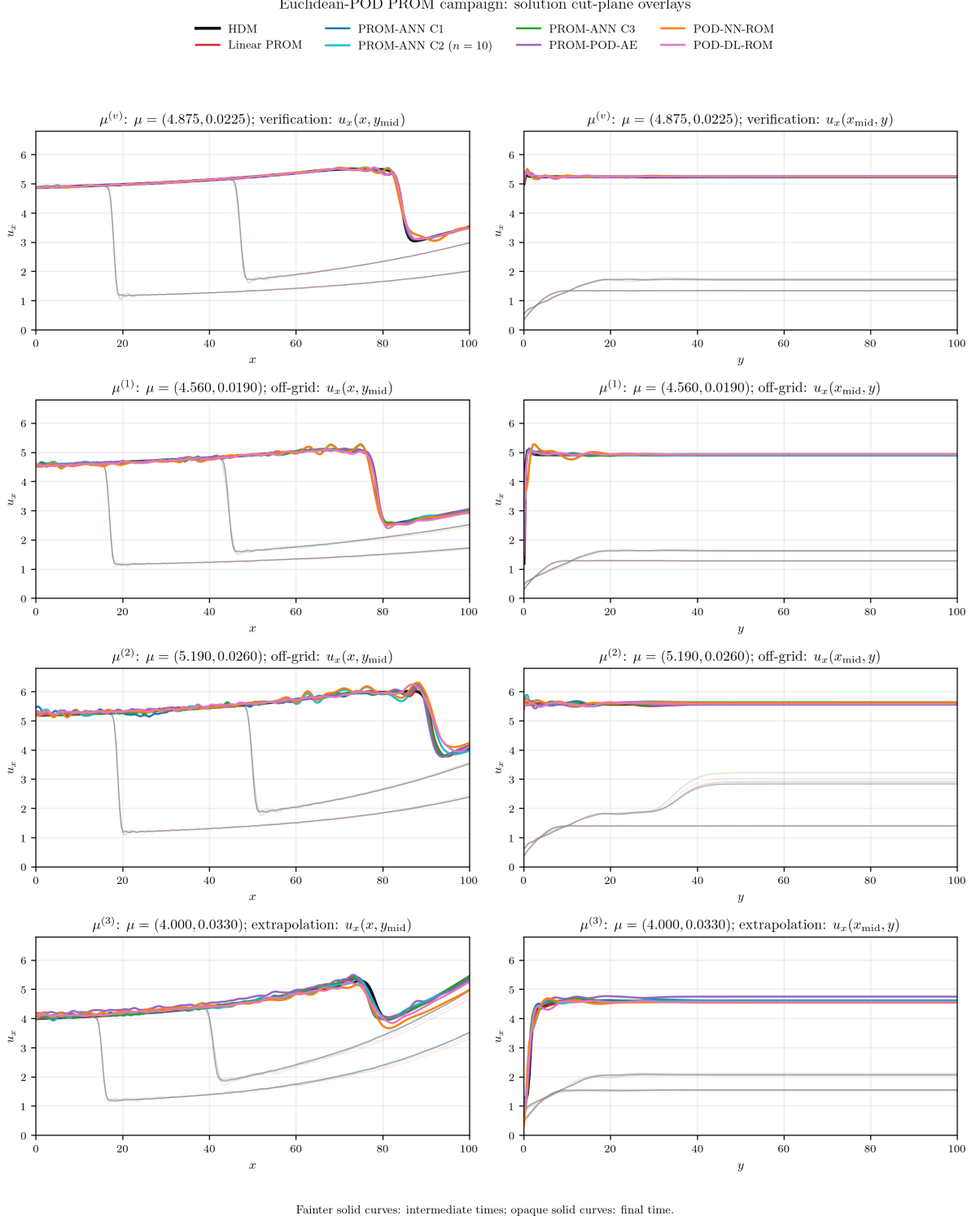


Figure 5: Final-time solution cut-plane overlays for the Euclidean-POD PROM campaign. Fainter solid curves show two intermediate times. Every intrusive curve uses the full residual; the two non-intrusive curves are direct reduced coordinate predictions.

6.2 Coefficient diagnostics

Table 5 and Fig. 6 compare each complete coordinate history with its 151-coordinate linear-PROM counterpart. The vertical line marks the $n = 10$ primary/secondary split used by

PROM-ANN. The direct maps are kept in the same figures and tables as the intrusive models, with an explicit separator, because all use the same linear-PROM coordinate teacher in this campaign.

Table 5: Relative coefficient-history errors e_q against the corresponding linear PROM. The horizontal rule separates intrusive and non-intrusive models, consistently with Table 4.

Model	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$
Linear PROM	0.000	0.000	0.000	0.000	0.000
PROM-ANN Case 1	0.324	0.751	1.391	0.822	3.304
PROM-ANN Case 2 ($n = 10$)	2.653	4.346	3.956	3.652	6.234
PROM-ANN Case 3	0.222	0.710	0.798	0.577	2.644
PROM-POD-AE ($n_z = 10$)	0.316	0.804	1.290	0.803	3.664
POD-NN-ROM	2.713	4.378	4.494	3.861	10.214
POD-DL-ROM ($n_z = 10$)	1.811	2.611	3.031	2.484	5.127

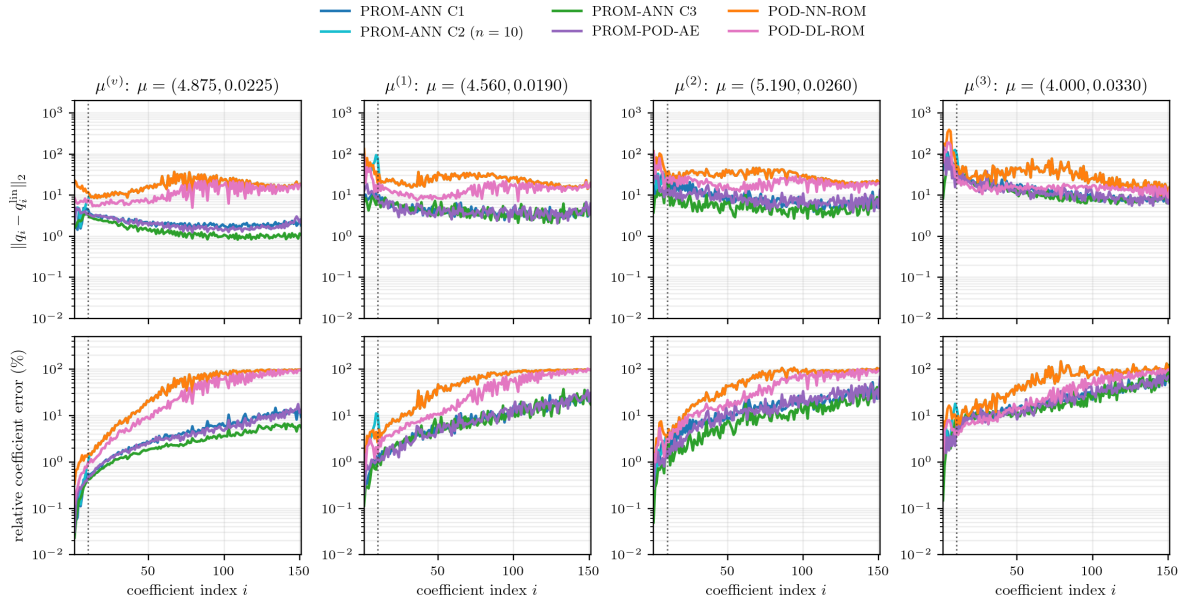


Figure 6: Per-coefficient absolute (top) and relative (bottom) errors against the 151-coordinate linear PROM. All panels in each row share the same logarithmic ordinate range. The dotted vertical line marks coefficient 10; all model curves are solid.

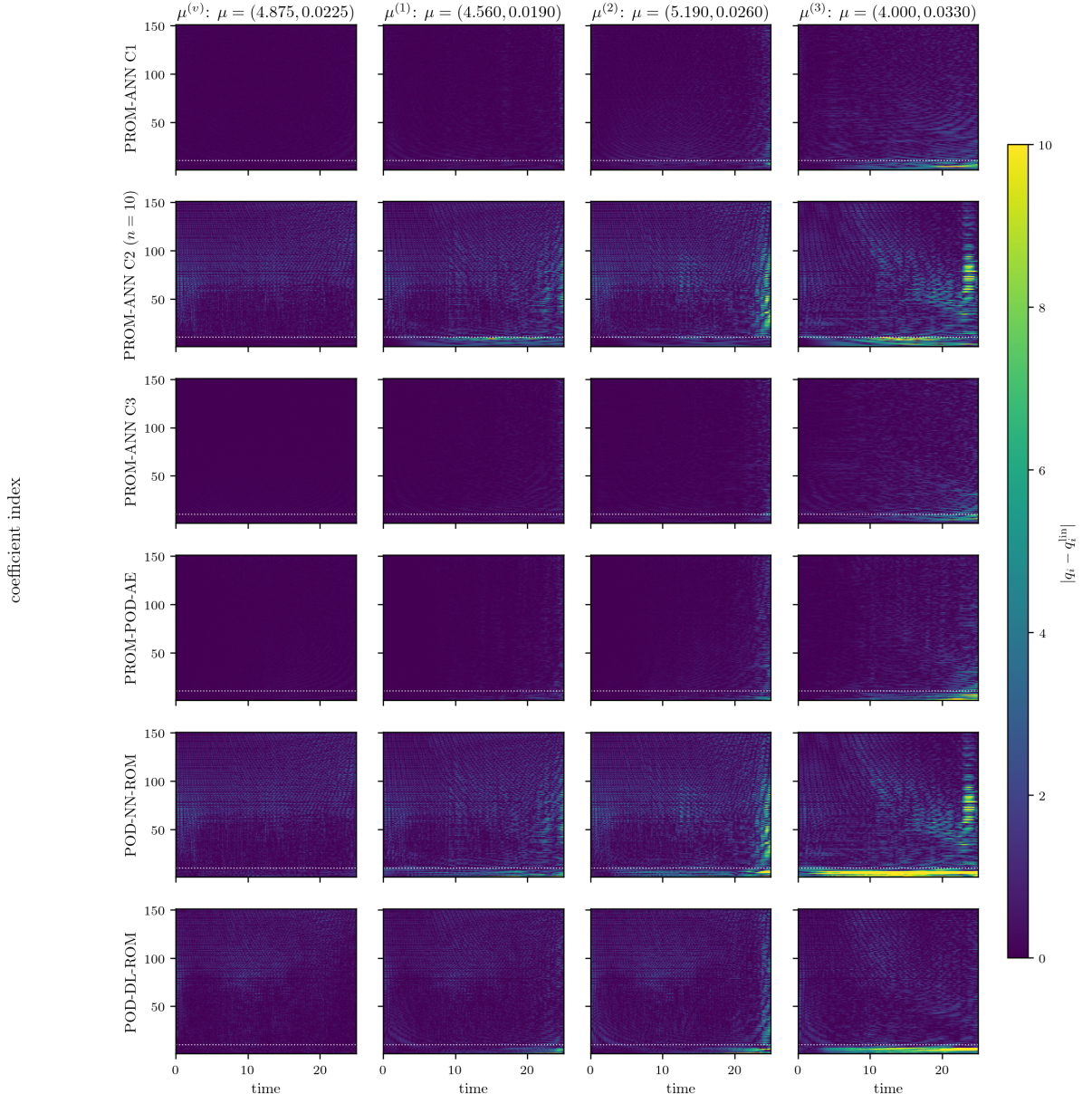


Figure 7: Absolute coefficient errors over time. Complete model names distinguish the intrusive PROM-POD-AE model from the non-intrusive POD-DL-ROM. The dotted white line marks the $n = 10$ primary split.

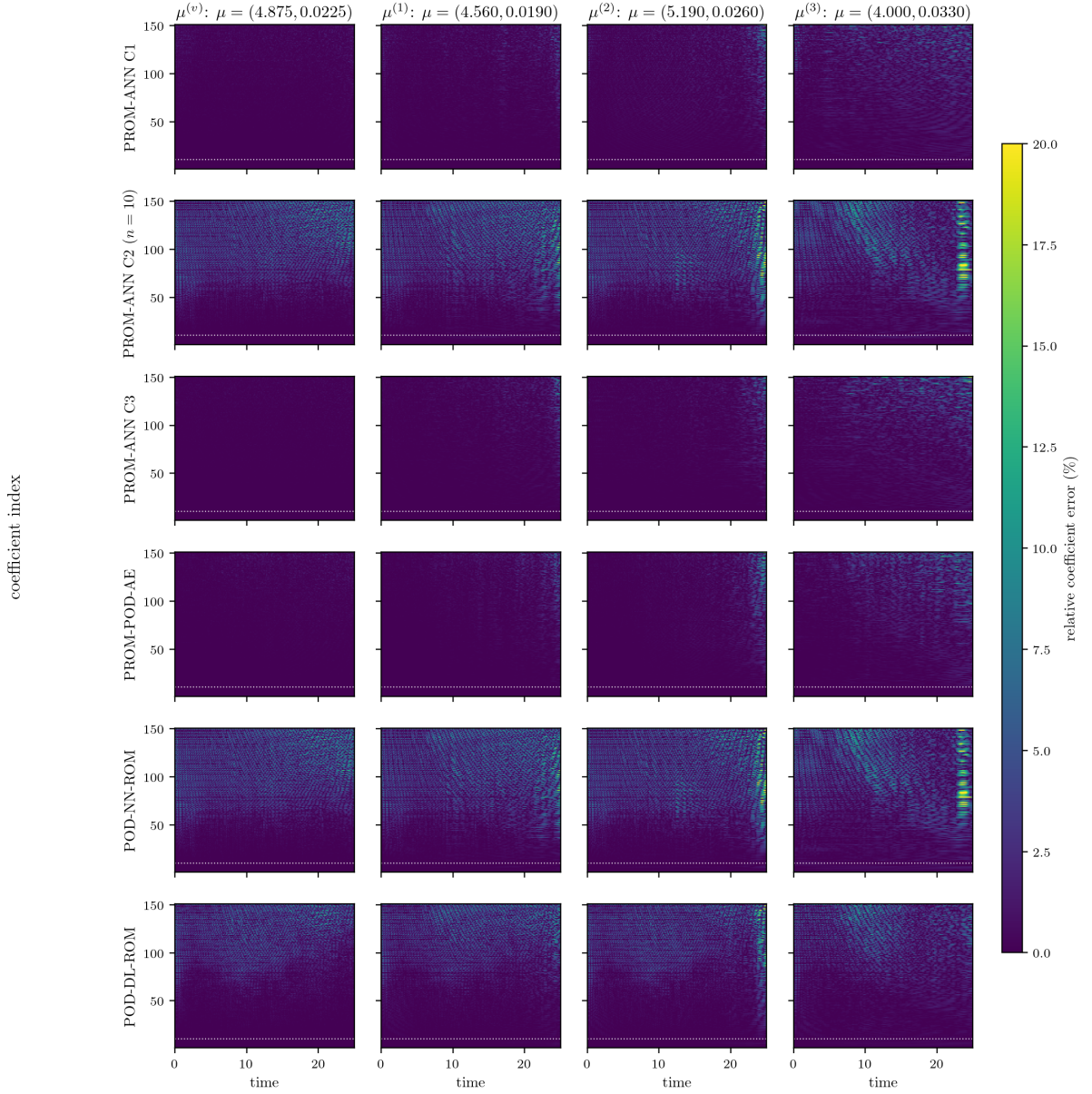


Figure 8: Relative coefficient errors over time, using the same ordering and coordinate split as Fig. 7.

6.3 Case-2 solved-dimension sweep

The shared master map permits an interpolation between a direct parameter-time regression and the full-coordinate linear PROM. Table 6 and Fig. 9 show that increasing n reduces state error, with the sharpest improvement between $n = 50$ and $n = 100$. This records the balance in this controlled Case-2 setting between coordinates supplied by the ANN and coordinates corrected by the residual.

Table 6: Case-2 state errors as the number of residual-solved primary coordinates is varied. The $n = 0$ and $n = 151$ endpoints are respectively the shared-map POD-NN-ROM and the linear PROM.

Solved n	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$
0 (POD-NN-ROM)	1.187	1.918	1.989	1.698	4.351
10	1.163	1.900	1.758	1.607	2.696
20	1.158	1.556	1.665	1.460	2.201
30	1.154	1.529	1.601	1.428	2.178
50	1.161	1.506	1.486	1.384	2.036
100	0.785	0.958	0.863	0.869	1.183
151 (linear PROM)	0.410	0.450	0.463	0.441	0.850

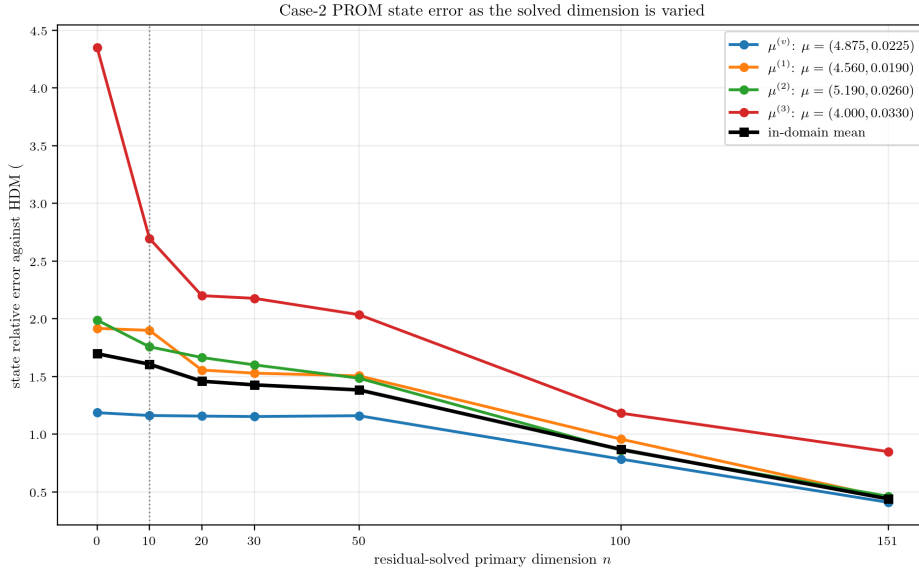


Figure 9: Case-2 state error versus the solved primary dimension. The black curve is the in-domain mean and the dotted line marks the baseline $n = 10$ split.

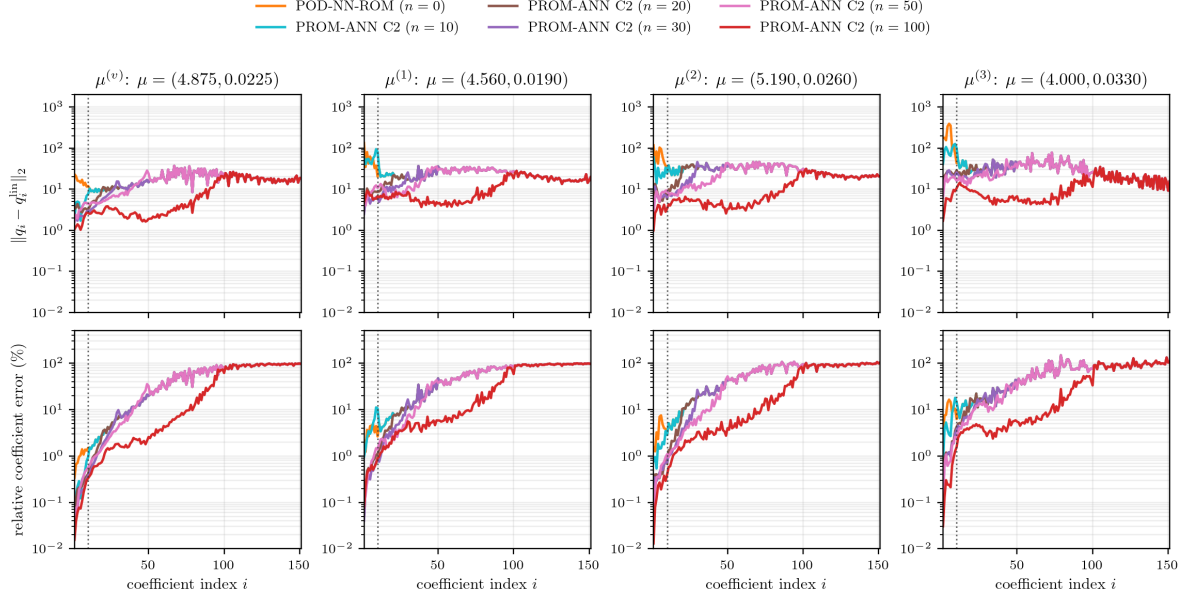


Figure 10: Case-2 coefficient trajectories in the shared-master-map sweep. Top: absolute error against linear PROM. Bottom: the corresponding relative error. The $n = 151$ linear-PROM reference has zero coefficient error and is therefore omitted.

6.4 Controlled Case-2 tail perturbation

To test the transfer mechanism independently of a regression magnitude, we start from the linear-PROM secondary history $\bar{\mathbf{q}}^{\text{lin}}$ and add a perturbation in the direction of the actual master-map secondary error:

$$\bar{\mathbf{q}}_\epsilon = \bar{\mathbf{q}}^{\text{lin}} + \frac{\epsilon}{100} \frac{\hat{\bar{\mathbf{q}}} - \bar{\mathbf{q}}^{\text{lin}}}{\|\hat{\bar{\mathbf{q}}} - \bar{\mathbf{q}}^{\text{lin}}\|_F} \|\bar{\mathbf{q}}^{\text{lin}}\|_F. \quad (58)$$

For each prescribed percentage ϵ , Case 2 solves only the first ten coordinates. Thus the diagnostic uses the observed ANN-error direction but does not conflate its effect with a change of learned model. Table 7 and Fig. 11 show a monotone transfer from secondary-tail error to the recovered primary block. The amount of transfer is parameter dependent, so a tail loss alone does not imply a pointwise primary-coordinate bound.

Table 7: Primary-coordinate error induced by a controlled secondary-tail perturbation in Case 2 with $n = 10$. Each entry is relative to the corresponding linear-PROM primary history.

Imposed tail error (%)	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	$\mu^{(3)}$
0	0.001	0.001	0.001	0.000
1	0.131	0.095	0.060	0.098
3	0.382	0.283	0.169	0.291
5	0.617	0.469	0.265	0.480
10	1.133	0.922	0.453	0.940
15	1.544	1.364	0.602	1.389
20	1.856	1.801	0.773	1.839
30	2.281	2.702	1.413	2.778
50	4.034	4.949	4.208	5.044

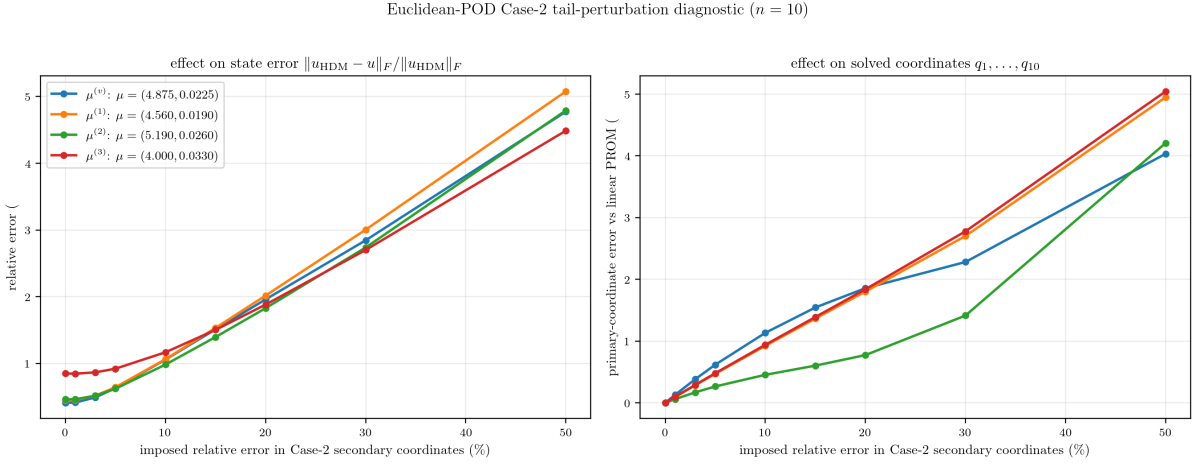


Figure 11: Euclidean-POD Case-2 sensitivity to a prescribed secondary-tail error, restricted to perturbations up to 50%. Left: state error against the HDM. Right: error in the ten residual-solved primary coordinates against linear PROM.

7 Case-2 Accuracy Recovery Before Data Enrichment

7.1 State accuracy with the unchanged baseline network

We now assess the residual-adaptive construction of Section 4 under the dedicated protocol of Section 5.5. The notation “Case 2 + $\mathbf{B}_3$ ” denotes three directions rebuilt at every time step, not a fixed offline basis. All learned models in this comparison retain their baseline training data; enrichment is examined separately in the next section.

Table 8: Common-initialization Sherlock comparison: state-history error e_u (%). Every learned model uses baseline training data. The mean averages the three in-domain points; the final column is extrapolatory. The linear PROM is the reference for the fixed 151-mode basis.

Model	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$
Linear PROM (151 modes)	0.410	0.450	0.463	0.441	0.850
PROM-ANN Case 2	1.162	1.947	1.743	1.617	2.643
PROM-ANN Case 2 + $\mathbf{B}_3$	0.412	0.446	0.483	0.447	0.850
PROM-ANN Case 1	0.418	0.543	0.736	0.566	1.673
PROM-ANN Case 3	0.408	0.512	0.563	0.494	1.521

Table 8 shows a reduction of the mean in-domain state error from 1.617% for uncorrected Case 2 to 0.447% for the adaptive variant, a 72.4% reduction. The latter is close to the linear-PROM mean of 0.441%. At the extrapolatory parameter the error decreases from 2.643% to 0.850%, also close to the linear reference. These changes require neither new coefficient training data nor additional network fitting. They result from allowing three residual-selected secondary patterns to be corrected together with the ten primary coordinates.

The appropriate baseline in this statement is 1.617%, not the 1.607% mean of Table 4: the former comes from the matched initial condition (55). Keeping both results explicit prevents an initialization change from being attributed to the correction space. The corrected method improves on Cases 1 and 3 in mean error and at extrapolation, but not at every parameter: Case 3 has a slightly smaller state error at verification. Likewise, the linear PROM is not a strict HDM-error lower bound; at $\mu^{(1)}$ the corrected trajectory has a slightly smaller HDM error than that reference.

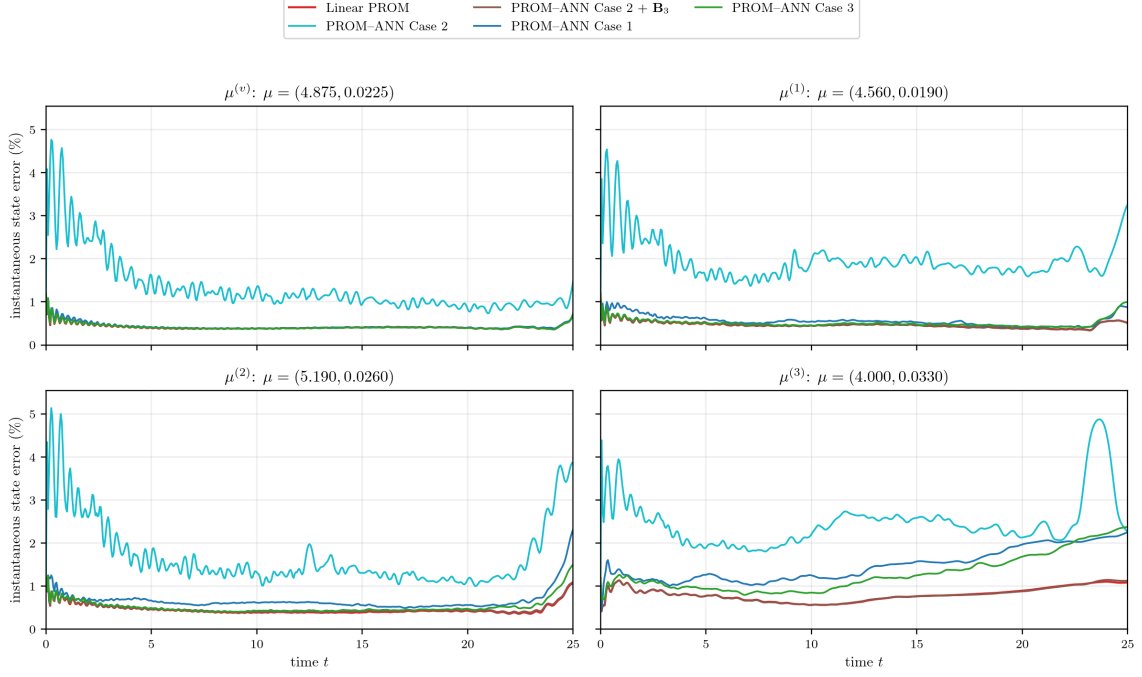


Figure 12: Instantaneous relative state errors $100 \|\mathbf{u}_{\text{HDM}}(t) - \mathbf{u}_{\text{ROM}}(t)\|_2 / \|\mathbf{u}_{\text{HDM}}(t)\|_2$ for the common-initialization comparison. Panels follow the four reporting parameters and share the same ordinate range. All model curves are solid. These time-resolved errors complement, rather than replace, the space-time norm in Table 8.

7.2 Recovery of the full coordinate trajectory

Table 9: Common-initialization Sherlock comparison: complete 151-coordinate trajectory error e_q (%) against the corresponding linear PROM. Means and parameter ordering match Table 8.

Model	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$
Linear PROM (151 modes)	0.000	0.000	0.000	0.000	0.000
PROM-ANN Case 2	2.655	4.454	3.926	3.678	6.104
PROM-ANN Case 2 + $\mathbf{B}_3$	0.087	0.266	0.291	0.215	0.441
PROM-ANN Case 1	0.326	0.749	1.393	0.823	3.303
PROM-ANN Case 3	0.225	0.712	0.799	0.579	2.644

The mean coefficient error decreases from 3.678% to 0.215% (Table 9). At extrapolation it decreases from 6.104% to 0.441%. The corrected mean is smaller than the 0.823% and 0.579% of Cases 1 and 3, respectively. Thus the state-error improvement is accompanied by recovery of the linear-PROM coordinate history, rather than only by a fortuitous cancellation in the HDM state norm. Figure 13 resolves the errors by coordinate. Although only three additional amplitudes are solved, their directions can change all 141 secondary coordinates.

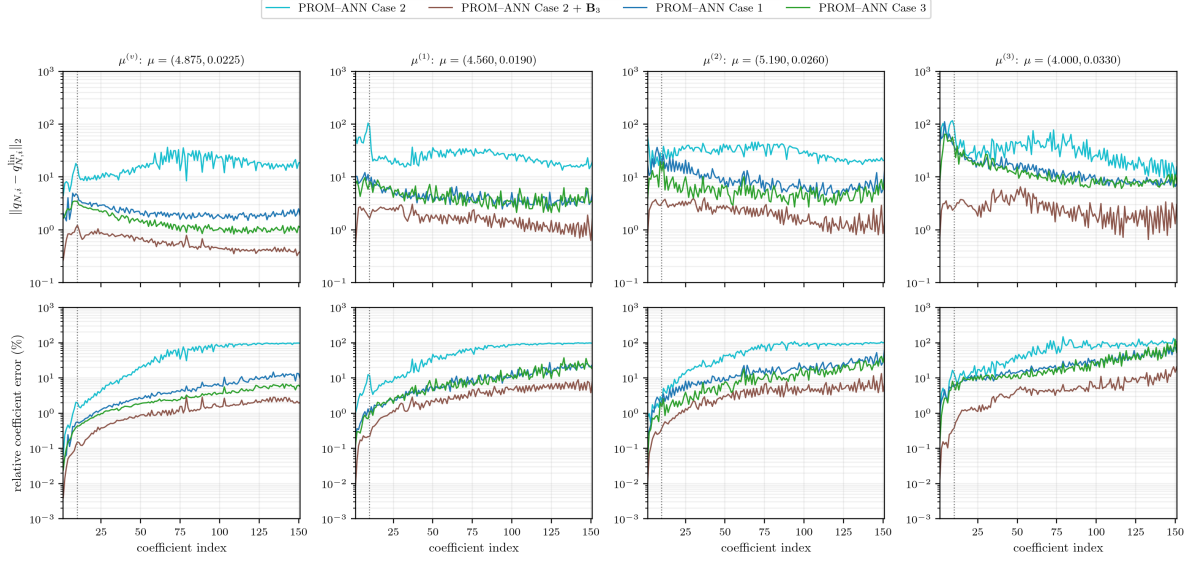


Figure 13: Common-initialization comparison: per-coordinate absolute trajectory error (top) and relative trajectory error in percent (bottom) against the linear PROM. All panels in each row share logarithmic ordinate limits computed from the plotted data. The dotted vertical line marks the ten primary coordinates; every model curve is solid. The zero-error linear-PROM reference is omitted from the logarithmic plots.

7.3 Measured online cost and scope of the result

Table 10: Measured full-trajectory online times (s) on Sherlock with 24 numerical threads. Each entry is one measurement, not an average of repeated runs. The mean averages the three in-domain parameters. Direction construction is included for Case 2 + $\mathbf{B}_3$; timing boundaries are specified in Section 5.5.

Model	$\mu^{(v)}$	$\mu^{(1)}$	$\mu^{(2)}$	Mean	$\mu^{(3)}$
PROM-ANN Case 2	116.7	84.5	83.9	95.0	88.2
PROM-ANN Case 2 + $\mathbf{B}_3$	199.8	162.6	161.9	174.8	165.3
PROM-ANN Case 1	463.8	430.0	497.3	463.7	432.9
PROM-ANN Case 3	539.6	459.1	436.3	478.3	438.6

The additional accuracy is not free. The mean time increases from 95.0 s for Case 2 to 174.8 s for the corrected method, a factor of 1.84. The measured direction/tangent-construction portion alone averages 68.0 s over the in-domain points. It would therefore be misleading to estimate the overhead only from the change from ten to thirteen unknowns. Nevertheless, in this implementation the corrected method takes less time than Cases 1 and 3, whose corresponding means are 463.7 s and 478.3 s. Their mean-time ratios to the corrected method are 2.65 and 2.74, respectively. These are method-to-method cost ratios, not HDM-relative speed-ups. A single run per point does not quantify timing variability, so these measurements support the observed ordering rather than a precise statistical performance claim.

For these four parameters, a small online secondary correction substantially closes the baseline Case-2 accuracy gap without enriching its training set. This is an alternative use of com-

putation, not evidence that enrichment is unnecessary: the adaptive method spends additional work at every deployment, whereas enrichment changes the offline data and leaves the original online architecture intact. Their relative total cost depends on the number of future queries. No corrected/enriched combination or hyperreduced adaptive variant has been tested here, and no general state-error guarantee follows from this experiment.

8 Effect of PROM Coefficient-Data Enrichment

The nested campaign tests how much of the baseline discrepancy can be reduced by extending parameter-space coverage at fixed architecture. The Euclidean basis, network architectures, two parameter-validation trajectories, four online evaluation points, and full-residual PROM formulations are fixed. Only the number and location of linear-PROM coefficient-training trajectories change through the nested budgets 9, 9 + 8, 9 + 12, and 9 + 18.

8.1 Offline regression accuracy

Table 11 reports errors in the reduced coordinates used to train and select the networks. Every learned family has substantially smaller external parameter-validation error at 9 + 12 and 9 + 18 than at baseline. The progression is not strictly monotone for POD-DL-ROM, whose error changes from 0.433% at 9 + 8 to 0.453% at 9 + 12. The validation error of the master parameter-time map shared by Case 2 and POD-NN-ROM decreases from 4.261% to 1.607% with 9 + 8, to 1.137% with 9 + 12, and to 0.990% with 9 + 18. This monotone held-out improvement does not, however, imply monotone accuracy at a more distant extrapolation point. The four Euclidean master maps use the same wide SiLU architecture, raw MSE objective, external validation data, and random seed. Their selected checkpoints occur at epochs 1010, 1634, 1253, and 1731 for the baseline, 9 + 8, 9 + 12, and 9 + 18 data, respectively, under the same validation-based early-stopping rule; none is the result of a wall-time interruption.

Table 11: Training and held-out parameter-validation coefficient errors across the Euclidean-POD PROM enrichment ladder. The master map supplies both the Case-2 secondary tail and the POD-NN-ROM direct prediction.

Model	Baseline (9)		Nested (9+8)		Nested (9+12)		Nested (9+18)	
	train e_q (%)	val. e_q (%)	train e_q (%)	val. e_q (%)	train e_q (%)	val. e_q (%)	train e_q (%)	val. e_q (%)
PROM-ANN Case 1	1.422	3.590	0.830	1.208	0.545	0.883	0.504	0.634
Master POD-NN-ROM (Case 2 tail source)	2.888	4.261	0.882	1.607	0.821	1.137	0.577	0.990
PROM-ANN Case 3	0.572	3.482	0.255	0.610	0.287	0.582	0.284	0.452
PROM-POD-AE ($n_z = 10$)	0.194	2.673	0.164	1.192	0.099	0.913	0.144	0.488
POD-DL-ROM ($n_z = 10$)	1.843	2.981	0.224	0.433	0.359	0.453	0.211	0.249

8.2 Online state accuracy

Table 12 and Fig. 14 show a clear in-domain benefit. For Case 2 with $n = 10$, the mean state error falls from 1.607% to 0.702%, 0.602%, and 0.481% at 9 + 8, 9 + 12, and 9 + 18, respectively. The direct POD-NN-ROM endpoint follows the same in-domain trend, from 1.698% to 0.735%, 0.609%, and 0.484%. Thus 9 + 12 closes most, but not all, of their baseline gap; 9 + 18 brings both close to the fixed linear-PROM mean of 0.441%. Case 1, Case 3, and PROM-POD-AE already reach the linear-reference scale at 9 + 12. POD-DL-ROM varies only slightly from 9 + 8 to 9 + 12 in domain and improves again at 9 + 18.

Table 12: Online state errors across the nested Euclidean-POD PROM coefficient-data ladder. Values are percentages. Each in-domain mean averages $\mu^{(v)}, \mu^{(1)}, \mu^{(2)}$; $\mu^{(3)}$ is extrapolatory. The linear PROM is unchanged, and the horizontal rule separates intrusive residual-minimizing models from non-intrusive direct maps.

Model	Baseline (9)		Nested (9+8)		Nested (9+12)		Nested (9+18)	
	in-domain mean	$\mu^{(3)}$	in-domain mean	$\mu^{(3)}$	in-domain mean	$\mu^{(3)}$	in-domain mean	$\mu^{(3)}$
Linear PROM	0.441	0.850	0.441	0.850	0.441	0.850	0.441	0.850
PROM-ANN Case 1	0.566	1.673	0.460	1.129	0.442	0.853	0.439	0.850
PROM-ANN Case 2 ($n = 10$)	1.607	2.696	0.702	3.270	0.602	0.935	0.481	0.876
PROM-ANN Case 3	0.494	1.521	0.442	1.054	0.440	0.851	0.440	0.848
PROM-POD-AE ($n_z = 10$)	0.551	1.754	0.454	1.183	0.442	0.853	0.441	0.859
POD-NN-ROM	1.698	4.351	0.735	19.465	0.609	1.083	0.484	0.938
POD-DL-ROM ($n_z = 10$)	1.125	2.267	0.461	1.103	0.456	0.868	0.444	0.858

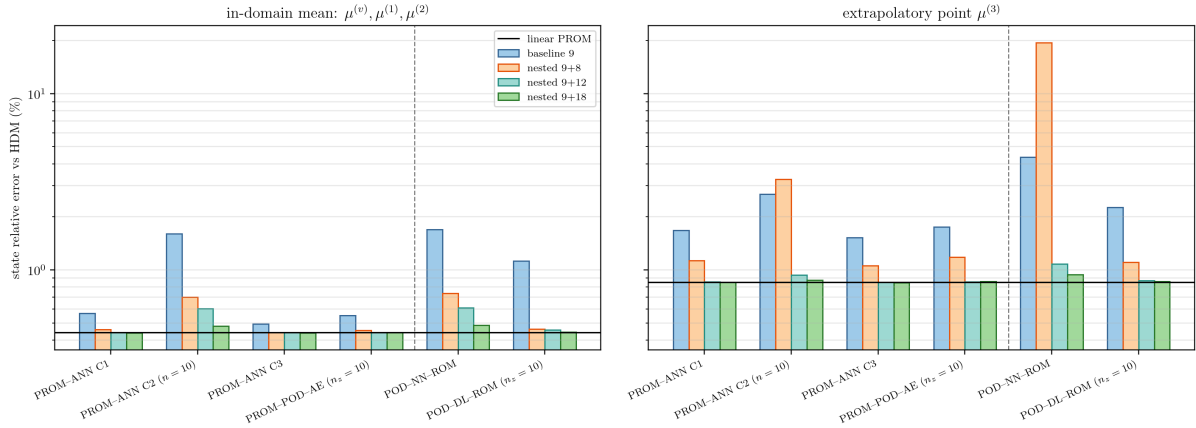


Figure 14: Effect of the four-level nested PROM coefficient-data ladder on online state error. Left: in-domain mean. Right: extrapolatory point. The two panels use exactly the same logarithmic ordinate limits, exposing the 9 + 8 extrapolation anomaly without hiding the near-linear results. The dashed vertical rule separates intrusive PROMs from non-intrusive ROMs; the solid black horizontal level is the fixed linear PROM.

The extrapolatory result at 9 + 8 is non-monotone and is retained rather than smoothed over. Case 2 changes from 2.696% at baseline to 3.270%, and POD-NN-ROM changes from 4.351% to 19.465%. Because these two models share the same parameter-time master map, while the other learned families improve at the same point, the degradation is specific to the extrapolation of that map rather than a uniform failure of the enriched data or the PROM solver. Residual minimization still provides substantial correction: using the same predicted 151-coordinate vector, Case 2 reduces the direct-map state error from 19.465% to 3.270% by solving the first ten coordinates. At 9 + 12, the Case-2 and POD-NN-ROM errors recover to 0.935% and 1.083%, respectively; 9 + 18 gives the slightly smaller values 0.876% and 0.938%.

The sampling geometry is consistent with this behaviour. The 9 + 8 subset contains a left-margin point at approximately (3.949, 0.0187) and a top-margin point at (4.783, 0.0325), but no sample in their upper-left intersection near $\mu^{(3)} = (4.000, 0.0330)$. The 9 + 12 subset adds (3.980, 0.03173), coincident with the recovery observed for all learned families at $\mu^{(3)}$. The subsequent 9 + 18 additions yield only modest further extrapolatory changes. This geometric evidence is consistent with the interpretation that counting margin points alone is insufficient. It does not prove causality, nor establish that the particular 9 + 8 deterioration is independent

of optimization seed or stopping time.

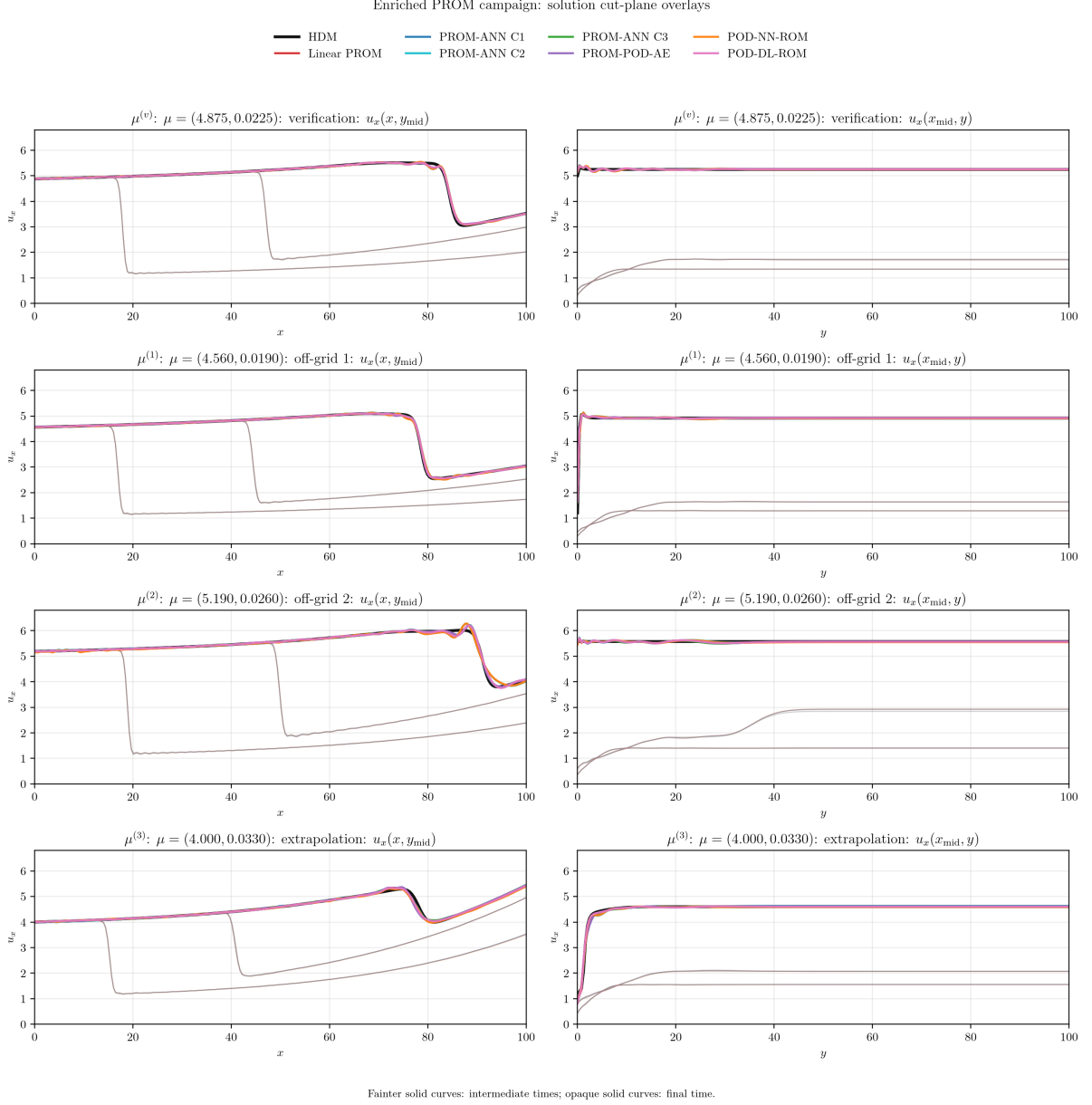


Figure 15: 9 + 18 Euclidean-POD PROM solution cut-plane comparison against the HDM. The model order, colours, state range, and time slices are identical to the baseline Fig. 5; all curves are solid.

8.3 Coefficient diagnostics

The coefficient results localize the non-monotone state behaviour before the online reconstruction. As shown in Table 13, the Case-2 extrapolatory coefficient error changes from 6.234% to 7.387% at 9+8, while the shared direct POD-NN-ROM error changes from 10.214% to 45.677%. The latter therefore already contains a severe coefficient-space extrapolation error. At 9 + 12, these values recover to 0.987% and 1.675%, and at 9 + 18 they fall further to 0.589% and 1.005%. The state and coefficient evidence consistently identify the master-map extrapolation, rather than the Case-2 residual solve, as the source of the early-subset anomaly.

Table 13: Online coefficient-history errors against the fixed $n_{\text{tot}} = 151$ linear PROM across the nested data ladder. Values are relative Frobenius percentages over all coefficients and time steps. The horizontal rule separates intrusive and non-intrusive models.

Model	Baseline (9)		Nested (9+8)		Nested (9+12)		Nested (9+18)	
	in-domain mean	$\mu^{(3)}$	in-domain mean	$\mu^{(3)}$	in-domain mean	$\mu^{(3)}$	in-domain mean	$\mu^{(3)}$
Linear PROM	0.000	0.000	0.000	0.000	0.000	0.000	0.000	0.000
PROM-ANN Case 1	0.822	3.304	0.317	1.571	0.150	0.257	0.122	0.208
PROM-ANN Case 2 ($n = 10$)	3.652	6.234	1.361	7.387	1.055	0.987	0.573	0.589
PROM-ANN Case 3	0.577	2.644	0.133	1.282	0.090	0.155	0.064	0.137
PROM-POD-AE ($n_z = 10$)	0.803	3.664	0.329	2.004	0.145	0.237	0.130	0.283
POD-NN-ROM	3.861	10.214	1.438	45.677	1.069	1.675	0.579	1.005
POD-DL-ROM ($n_z = 10$)	2.484	5.127	0.354	1.819	0.394	0.451	0.189	0.331

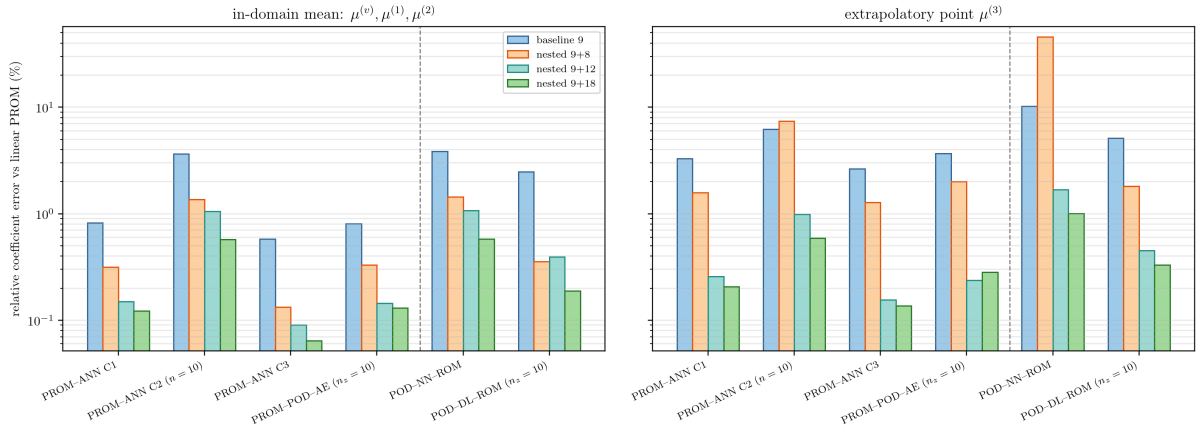


Figure 16: Effect of coefficient-data enrichment on global coefficient error. The logarithmic ordinate keeps both the $9 + 8$ POD-NN-ROM extrapolation error and the near-linear $9 + 18$ results visible. The dashed vertical rule separates intrusive PROMs from direct ROM maps.

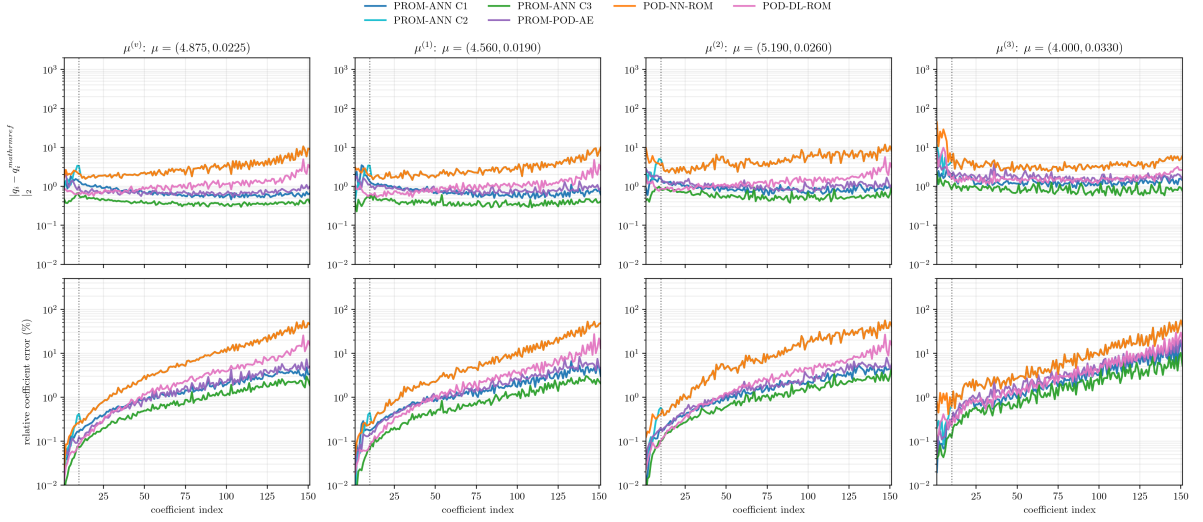


Figure 17: 9+18 per-coefficient absolute and relative trajectory errors against the linear PROM. The plotted set and fixed ordinate limits are identical to the baseline Fig. 6. The dotted vertical line marks the ten-coordinate primary/secondary split.

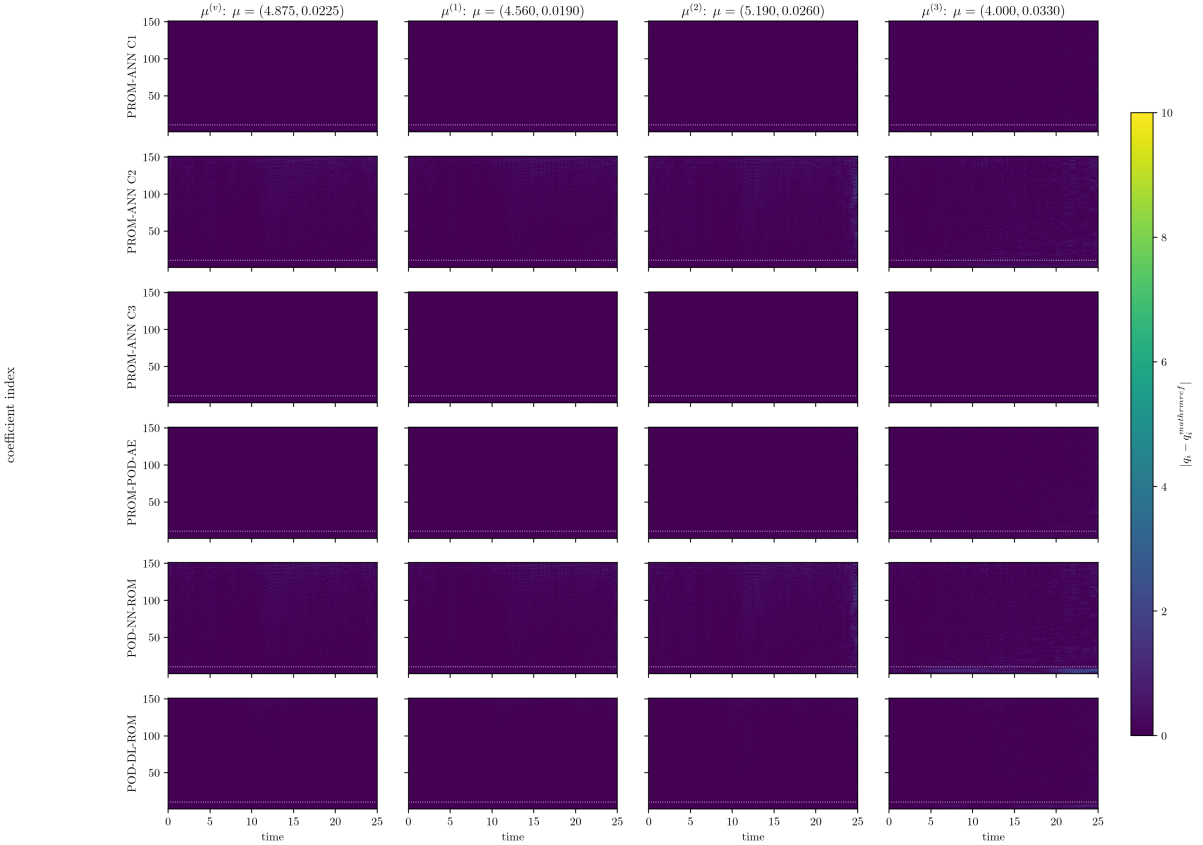


Figure 18: 9 + 18 time-by-coefficient absolute errors against the linear PROM. Rows contain the four intrusive models followed by POD-NN-ROM and POD-DL-ROM; the colour range matches the baseline diagnostic.

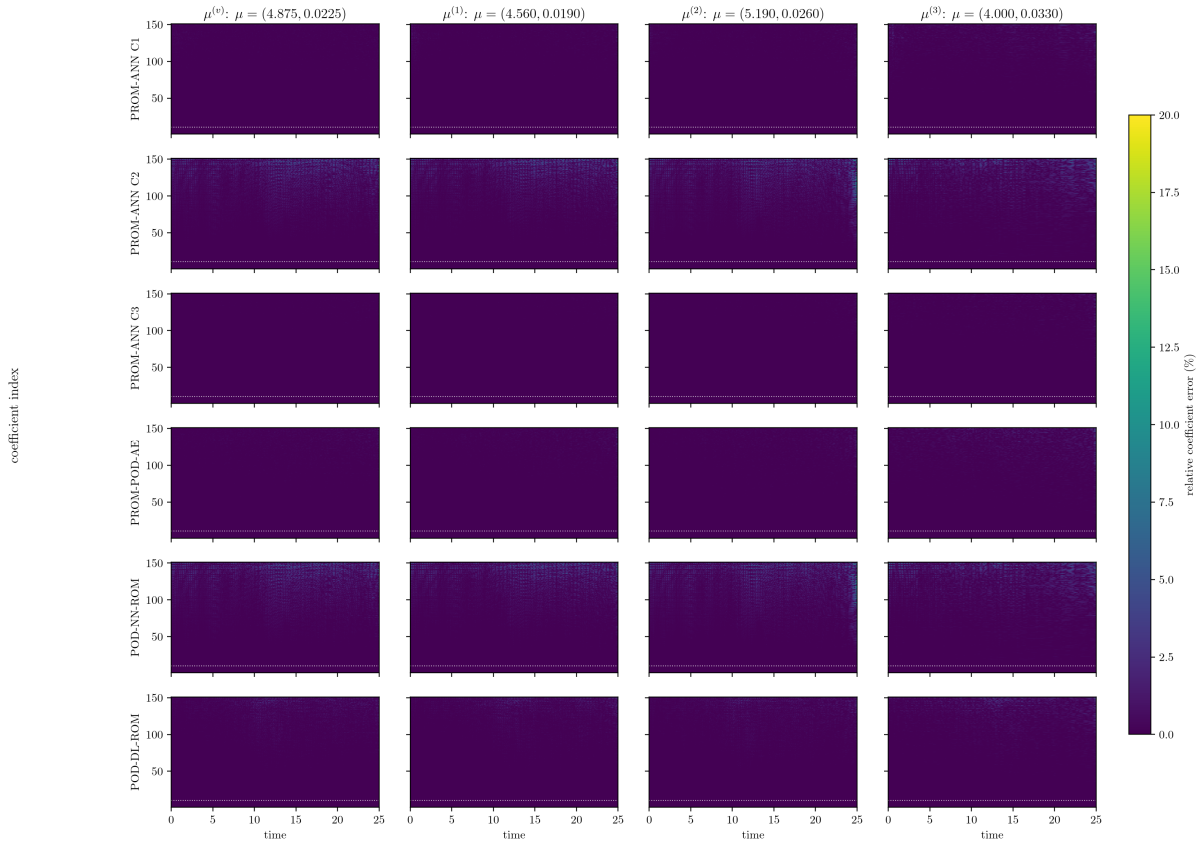


Figure 19: 9 + 18 time-by-coefficient relative errors, with the same common 0–20% colour range and model ordering as the baseline diagnostic.

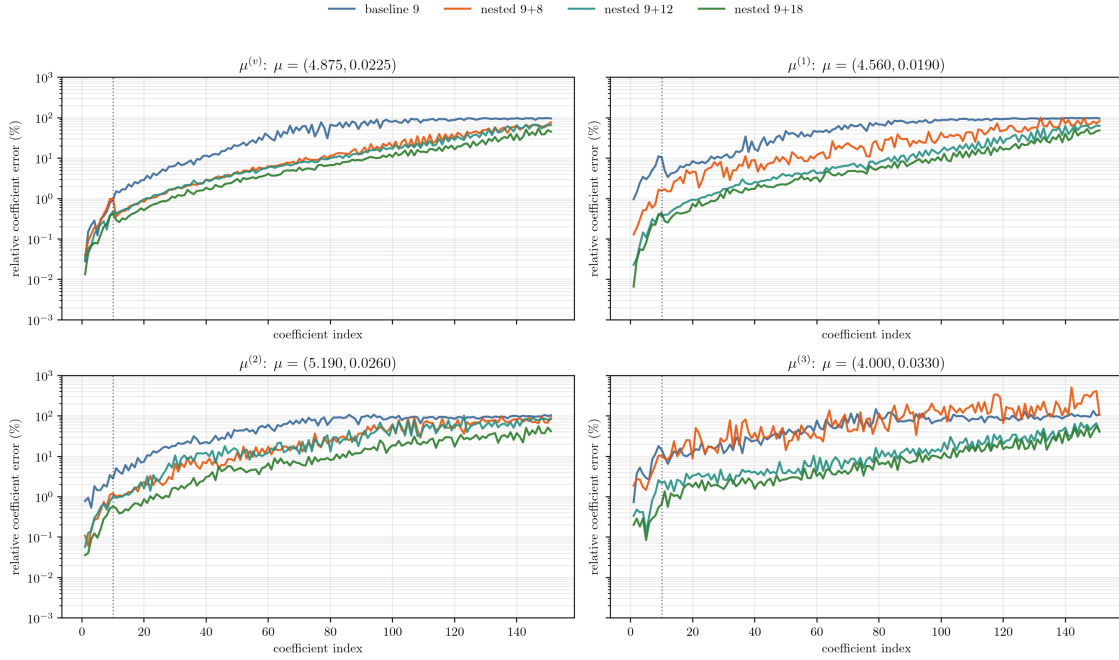


Figure 20: Case-2 per-coefficient relative error for the baseline, 9 + 8, 9 + 12, and 9 + 18 master maps. Each panel is one online evaluation point. The dotted vertical line separates the ten residual-solved primary coordinates from the 141 ANN-injected secondary coordinates.

The nested experiment therefore supports two distinct conclusions. First, coefficient-data coverage is an important limitation in domain: $9+8$ substantially improves every model, $9+12$ places four of the six learned models at the linear-reference state-error scale, and $9+18$ also closes most of the remaining Case-2 and POD-NN-ROM gap. Second, enrichment size alone does not control extrapolation. The $9+8$ subset reduces held-out parameter validation error while worsening the corner extrapolation; adding four well-placed trajectories at $9+12$ reverses that failure. The location of the added trajectories is therefore as important as their count.

9 Conclusions

The Euclidean-POD PROM campaign gives a common full-residual comparison of three learned closures, an intrusive decoder, and two direct reduced-coordinate maps. The linear PROM provides an accuracy reference in the fixed 151-coordinate trial representation. Case 3 and PROM-POD-AE retain close in-domain state accuracy while solving a low-dimensional online variable. The Case-2/POD-NN shared-map study gives a precise interpretation of intrusive correction: solving progressively more coordinates through the residual moves the result from the direct-map endpoint toward linear PROM, but the prescribed tail can still transfer its error into the solved block.

The residual-adaptive extension provides a different response to that limitation. With the unchanged baseline network and three additional secondary amplitudes, the common-initialization comparison reduces the mean in-domain state error from 1.617% to 0.447%, close to the linear reference. Its measured mean online cost is 1.84 times that of uncorrected Case 2, but lower than the measured costs of Cases 1 and 3. The construction is a local residual-based correction, not an HDM-error minimization principle or a guarantee of improvement for arbitrary data.

The nested enrichment study shows that $9+12$ already reaches the linear-reference scale for four of the six learned models and removes the localized master-map extrapolation failure observed at $9+8$. Increasing to $9+18$ remains useful for Case 2 and POD-NN-ROM in domain and brings all learned models close to the linear-PROM reference at the reported points. The comparison demonstrates that the geometry of added samples matters in addition to their number.

The results intentionally do not mix PROM evidence with an unavailable HPROM data set. The ECM derivation establishes the required model-specific construction: an HPROM rule must be built from each learned trial map and its tangent, rather than borrowed from a linear HPROM or from a different closure. The next HPROM campaign can therefore use the same reporting order, tables, and model comparisons as the PROM study.

A LSPG-Sensitive Basis Exploration

Before adopting the Euclidean POD basis used in the main text, we explored an M -regularized basis metric intended to emphasize sensitivity of the LSPG normal equations to reduced-coordinate perturbations. The objective was to reduce transfer from a learned secondary block to residual-solved primary coordinates without degrading the linear reference. In the present Burgers campaign, controlled tests did not establish a systematic improvement sufficient to justify making that metric the main experimental variable. For clarity and reproducibility, every reported result therefore uses the standard Euclidean POD basis of (4). The metric-aware construction is retained as an exploratory direction, not as a contribution or as a source of the numerical comparisons in this paper. The historical LSPG-sensitive enrichment campaign also selected an ELU master map, whereas the Euclidean campaign uses SiLU. Its earlier extrapolation trend is therefore not an architecture-controlled metric comparison and cannot be attributed to the basis choice alone.

References

- [1] Athanasios C Antoulas. *Approximation of large-scale dynamical systems*. SIAM, 2005.
- [2] Alfio Quarteroni, Gianluigi Rozza, et al. *Reduced order methods for modeling and computational reduction*, volume 9. Springer, 2014.
- [3] Joshua Barnett, Charbel Farhat, and Yvon Maday. Neural-network-augmented projection-based model order reduction for mitigating the Kolmogorov barrier to reducibility. *Journal of Computational Physics*, 492:112420, 2023.
- [4] Albert Cohen, Charbel Farhat, Yvon Maday, and Agustin Somacal. Nonlinear compressive reduced basis approximation for PDEs. *Comptes Rendus. Mécanique*, 351(S1):357–374, 2023.
- [5] Joubine Aghili, Hassan Ballout, Yvon Maday, and Christophe Prud’Homme. Nonlinear compressive reduced basis approximation: when Taylor meets Kolmogorov. *arXiv preprint arXiv:2601.13712*, 2026.
- [6] Joaquín Alberto Hernández, Manuel Alejandro Caicedo, and Alex Ferrer. Dimensional hyper-reduction of nonlinear finite element models via empirical cubature. *Computer Methods in Applied Mechanics and Engineering*, 313:687–722, 2017.
- [7] JA Hernández. A multiscale method for periodic structures using domain decomposition and ecm-hyperreduction. *Computer Methods in Applied Mechanics and Engineering*, 368:113192, 2020.
- [8] Charbel Farhat, Todd Chapman, and Philip Avery. Structure-preserving, stability, and accuracy properties of the energy-conserving sampling and weighting method for the hyper reduction of nonlinear finite element dynamic models. *International Journal for Numerical Methods in Engineering*, 102(5):1077–1110, 2015.
- [9] Sebastian Ares De Parga, Radek Tezaur, Carlos G. Hernández, and Charbel Farhat. Nonlinear projection-based model order reduction with machine learning regression for closure error modeling in the latent space. *Computer Methods in Applied Mechanics and Engineering*, 448:118443, 2026.
- [10] S Ares De Parga, Jose Raul Bravo, JA Hernández, Rubén Zorrilla, and Riccardo Rossi. Hyper-reduction for Petrov–Galerkin reduced order models. *Computer Methods in Applied Mechanics and Engineering*, 416:116298, 2023.
- [11] Stefania Fresca and Andrea Manzoni. Pod-dl-rom: Enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. *Computer Methods in Applied Mechanics and Engineering*, 388:114181, 2022.